Containerization and DevOps — Lab Report

Nakul (500121882)

May 2026

Contents

1 Containerization and DevOps **4**
1.1 SAP ID 4
1.2 About This Repository 4
1.3 Repository Structure 5
1.4 Theory Section 5
1.5 Lab Experiments 5

2 Lab Experiment 1 – Comparison of Virtual Machines and Containers **6**
2.1 Objective 6
2.2 Tools Used 6

3 Part A – Virtual Machine Setup **6**

4 Part B – Docker Container Setup **10**
4.1 Result 15
4.2 Navigation 17

5 Installing Docker on Windows & Making GitHub.io Live **17**

6 Part 1: Installing Docker on Windows **17**
6.1 Step 1: Install Docker Desktop for Windows 18
6.2 Step 2: Verify Docker Installation 18
6.3 Step 3: Run an Ubuntu Container 18
6.4 Step 4: Install Java Inside the Container (Example) 18

7 Part 2: Hosting a Website with GitHub Pages **19**
7.1 Step 1: Create a Repository 19
7.2 Step 2: Add Website Files 19
7.3 Step 3: Enable GitHub Pages 20
7.4 Step 4: Access the Live Website 20
7.5 Screenshot (Example) 20
7.6 Navigation 20

8 Lab 3 – Docker Images, NGINX & Flask Application Deployment **21**
8.1 Objective 21

9 Detailed Implementation Files **21**

10 Part A – Working with Official NGINX Image **22**

11 Part B – Custom NGINX Image (Ubuntu Based) **22**

12 Part C – Lightweight NGINX (Alpine Based) **23**

13 Part D – Deploying Custom HTML using Bind Mount **23**

14 Part E – Containerizing a Flask Web Application **23**

15 Overall Concepts Covered **24**

16 Result **24**

17 Conclusion **24**
17.1 Navigation 25

18 Lab 4: Docker Essentials **25**
18.1 Objective 25
18.2 Part 1: Containerizing Applications with Dockerfile 25
18.3 Part 2: Using .dockerignore 26
18.4 Part 3: Building Docker Images 27
18.5 Part 4: Running Containers 28
18.6 Part 5: Multi-stage Builds 28
18.7 Part 6: Publishing to Docker Hub 32
18.8 Part 7: Node.js Example 34
18.9 Part 8: Practice Exercises 36
18.10 Cleanup 36
18.11 Docker Commands Cheat Sheet 39
18.12 Key Takeaways 39
18.13 Navigation 39

19 Lab 5: Docker Volumes, Environment Variables, Monitoring & Networks **40**
19.1 Objective 40
19.2 Part 1: Docker Volumes – Persistent Data Storage 40
19.3 Part 2: Environment Variables 41
19.4 Part 3: Docker Monitoring 44
19.5 Part 4: Docker Networks 46
19.6 Part 5: Real-World Example (Multi-Container Setup) 48
19.7 Cleanup 49
19.8 Docker Commands Cheat Sheet 49
19.9 Key Takeaways 50
19.10 Navigation 50

20 Experiment 6A: Comparison of Docker Run and Docker Compose **51**
20.1 PART A – THEORY 51
20.2 PART B – PRACTICAL TASKS 52
20.3 PART C – CONVERSION TASKS (Theory) 56

20.4 PART D – DOCKERFILE WITH COMPOSE . . . 56

21 Experiment 6B: WordPress + MySQL using Compose 58
21.1 Docker Compose vs Swarm Comparison . . . 59
21.2 Key Takeaways . . . 62
21.3 Navigation . . . 62

22 Lab 7: CI/CD using Jenkins, GitHub & Docker Hub 62
22.1 1. Aim . . . 62
22.2 2. Objective . . . 62
22.3 3. What is CI/CD? . . . 63
22.4 4. What is Jenkins? . . . 63
22.5 5. Project Structure . . . 63
22.6 6. Application Code (Flask) . . . 63
22.7 7. Dockerfile . . . 64
22.8 8. Jenkinsfile (Pipeline) . . . 64
22.9 9. Jenkins Setup using Docker . . . 65
22.10 10. Jenkins Configuration . . . 66
22.11 11. Webhook Integration . . . 66
22.12 12. Execution Flow . . . 67
22.13 13. Key Concept: `withCredentials` . . . 67
22.14 14. Key Commands in Pipeline . . . 67
22.15 15. Observations & Screenshots . . . 68
22.16 16. Result . . . 68
22.17 Key Takeaways . . . 73
22.18 Navigation . . . 73

23 Experiment 9: Ansible – Execution Prompt 73
23.1 Objective . . . 73
23.2 Prerequisites . . . 73
23.3 Steps Performed . . . 74
23.4 Results . . . 81

24 Experiment 10: SonarQube – Static Code Analysis 82
24.1 1. Aim . . . 82
24.2 2. Problem Statement . . . 82
24.3 3. What is SonarQube? . . . 82
24.4 4. What Problems It Solves . . . 82
24.5 5. Key Terms (VERY IMPORTANT) . . . 82
24.6 6. Architecture (VERY IMPORTANT) . . . 82
24.7 7. Important Concept . . . 83
24.8 8. Token Authentication (IMPORTANT) . . . 83
24.9 9. Step-by-Step Flow . . . 83
24.10 10. Tool Comparison (Important Theory) . . . 86
24.11 11. Best Practices . . . 86
24.12 12. Final Summary . . . 86
24.13 FINAL UNDERSTANDING . . . 88

25 Experiment 11: Orchestration using Docker Compose & Docker Swarm 88
25.11. Aim 88
25.22. Problem Statement 88
25.33. Concepts 88
25.44. Prerequisites 89
25.55. Steps Performed 89
25.66. Key Observations 94
25.77. Quick Reference 95
25.88. Result 95
25.99. Navigation 96

26 Experiment 12: Container Orchestration using Kubernetes 96
26.11. Objective 96
26.22. Why Kubernetes over Docker Swarm? 96
26.33. Core Kubernetes Concepts 96
26.44. Prerequisites 97
26.55. Steps Performed 97
26.66. Swarm vs Kubernetes 101
26.77. Kubernetes Command Cheat Sheet 101
26.88. Result 102
26.99. Navigation 102

1 Containerization and DevOps

1.1 SAP ID

500121882

1.2 About This Repository

This repository contains theory notes and lab experiments for the subject **Containerization and DevOps**. It focuses on conceptual understanding and hands-on implementation.

1.2.1 Important Links

- **GitHub Repository**: Nakul71/containerization-and-devops
- **GitHub Pages**: https://nakul71.github.io/containerization-and-devops/

Key areas covered:

- Virtualization
- Containerization
- Docker
- Kubernetes
- CI/CD with Jenkins
- Configuration Management with Ansible
- Code Quality with SonarQube

1.3 Repository Structure

```
Containerization-DevOps/
|-- Theory/
|   |-- Class1/ ... Class14/
`-- Lab/
    |-- Lab-1/
    |-- Lab2/
    |-- Lab3/
    |-- Lab4/
    |-- Lab5/
    |-- Lab6/
    |-- Lab7/
    |-- Lab9/
    `-- Lab10/
```

1.4 Theory Section

Link to theory index:

- Go to Theory README

Direct links:

- Class 1 — Docker Basics
- Class 2 — Application Containerization
- Class 3 — Docker Engine API
- Class 4 — Multistage Docker Build
- Class 5 — Docker Volumes & Bind Mounts
- Class 6 — Docker Networking
- Class 7 — Docker Advanced Commands
- Class 8 Part 1 — Docker Compose
- Class 8 Part 2 — Docker Compose
- Class 9 Part 1 — Compose & Reverse Proxy
- Class 9 Part 2 — Compose & Reverse Proxy
- Class 10 — Portainer GUI Management
- Class 11 — Introduction to Kubernetes
- Class 12 — Kubernetes Review & Basics
- Class 13 — Apache Web App Deployment
- Class 14 — Git Basics

1.5 Lab Experiments

- Lab 1 — VM vs Container Comparison

- Lab 2 — Docker Installation & GitHub Pages
- Lab 3 — Docker Images, NGINX & Flask Deployment
- Lab 4 — Docker Essentials
- Lab 5 — Volumes, Env Vars, Monitoring, Networks
- Lab 6 — Docker Run vs Docker Compose
- Lab 7 — CI/CD using Jenkins, GitHub & Docker Hub
- Lab 9 — Automating Server Management with Ansible
- Lab 10 — Static Code Analysis with SonarQube

2 Lab Experiment 1 – Comparison of Virtual Machines and Containers

2.1 Objective

- Understand differences between Virtual Machines and Containers.
- Install and configure VM using VirtualBox and Vagrant.
- Install and configure Containers using Docker on WSL.
- Deploy Nginx web server in both environments.
- Observe system resource utilization.

2.2 Tools Used

- VirtualBox
- Vagrant
- Ubuntu
- WSL
- Docker
- Nginx
- VS Code

---

3 Part A – Virtual Machine Setup

3.0.1 Step 1 – Download VirtualBox

3.0.2 Step 2 – Install Vagrant

3.0.3 Step 3 – Initialize Vagrant

```
vagrant init hashicorp/bionic64
```

3.0.4 Step 4 – Start VM

```
vagrant up
```

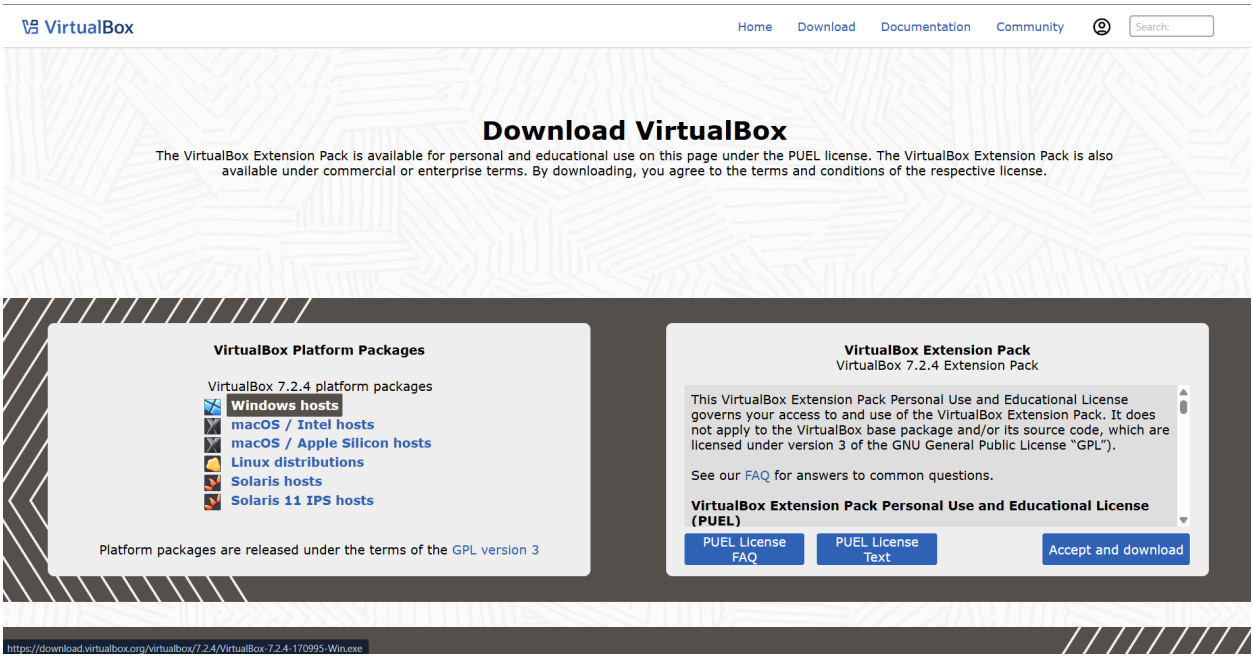


Figure 1: Step1

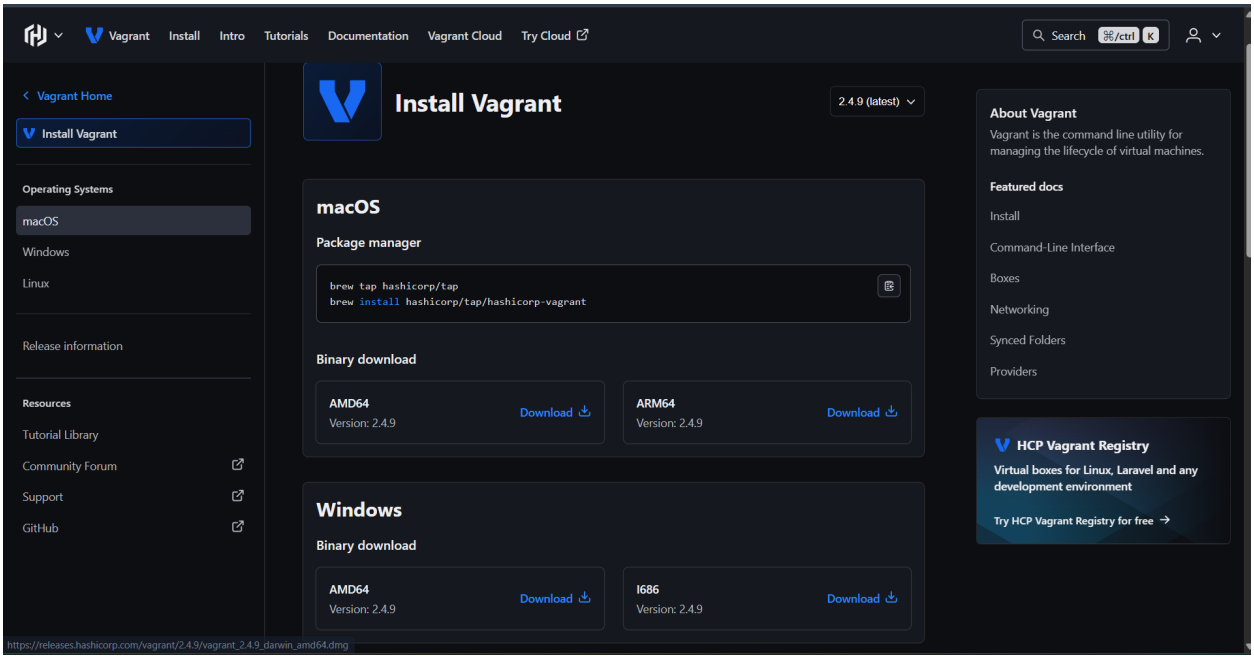


Figure 2: Step2

```
PS C:\Users\nakul\OneDrive\Desktop\Cloud\sem 6 Lab> vagrant init hashicorp/bionic64
A `Vagrantfile` has been placed in this directory. You are now
ready to `vagrant up` your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
`vagrantup.com` for more information on using Vagrant.
PS C:\Users\nakul\OneDrive\Desktop\Cloud\sem 6 Lab>
```

Figure 3: Step3

```
PS C:\Users\nakul\OneDrive\Desktop\Cloud\sem 6 Lab> vagrant init hashicorp/bionic64
A `Vagrantfile` has been placed in this directory. You are now
ready to `vagrant up` your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
`vagrantup.com` for more information on using Vagrant.
PS C:\Users\nakul\OneDrive\Desktop\Cloud\sem 6 Lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtualbox
    default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/versions/1.0.282/providers/virtualbox/unknown/vagrant.box
    default:
==> default: Successfully added box 'hashicorp/bionic64' (v1.0.282) for 'virtualbox'!
==> default: Importing base box 'hashicorp/bionic64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'hashicorp/bionic64' version '1.0.282' is up to date...
==> default: Setting the name of the VM: sem6Lab_default_1769246846073_10543
Vagrant is currently configured to create VirtualBox synced folders with
the `SharedFoldersEnableSymlinksCreate` option enabled. If the Vagrant
guest is not trusted, you may want to disable this option. For more
information on this option, please refer to the VirtualBox manual:

  https://www.virtualbox.org/manual/ch04.html#sharedfolders

This option can be disabled globally with an environment variable:

  VAGRANT_DISABLE_VBOXSYMLINKCREATE=1

or on a per folder basis within the Vagrantfile:

  config.vm.synced_folder '/host/path', '/guest/path', SharedFoldersEnableSymlinksCreate: false
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
    default: Adapter 1: nat
==> default: Forwarding ports...
    default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
    default: SSH address: 127.0.0.1:2222
    default: SSH username: vagrant
    default: SSH auth method: private key
```

Figure 4: Step4

```
Vagrant is currently configured to create VirtualBox synced folders with
the `SharedFoldersEnableSymlinksCreate` option enabled. If the Vagrant
guest is not trusted, you may want to disable this option. For more
information on this option, please refer to the VirtualBox manual:

  https://www.virtualbox.org/manual/ch04.html#sharedfolders

This option can be disabled globally with an environment variable:

  VAGRANT_DISABLE_VBOXSYMLINKCREATE=1

or on a per folder basis within the Vagrantfile:

  config.vm.synced_folder '/host/path', '/guest/path', SharedFoldersEnableSymlinksCreate: false
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
    default: Adapter 1: nat
==> default: Forwarding ports...
    default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
    default: SSH address: 127.0.0.1:2222
    default: SSH username: vagrant
    default: SSH auth method: private key
    default: Warning: Connection reset. Retrying...
    default: Warning: Connection reset. Retrying...
    default:
    default: Vagrant insecure key detected. Vagrant will automatically replace
    default: this with a newly generated keypair for better security.
    default:
    default: Inserting generated public key within guest...
    default: Removing insecure key from the guest if it's present...
    default: Key inserted! Disconnecting and reconnecting using new SSH key...
==> default: Machine booted and ready!
==> default: Checking for guest additions in VM...
    default: The guest additions on this VM do not match the installed version of
    default: VirtualBox! In most cases this is fine, but in rare cases it can
    default: prevent things such as shared folders from working properly. If you see
    default: shared folder errors, please make sure the guest additions within the
    default: virtual machine match the version of VirtualBox you have installed on
    default: your host and reload your VM.
    default:
    default: Guest Additions Version: 6.0.10
    default: VirtualBox Version: 7.2
==> default: Mounting shared folders...
    default: C:/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab => /vagrant
PS C:\Users\nakul\OneDrive\Desktop\Cloud\sem 6 Lab>
```

Figure 5: Step5

3.0.5 Step 5 – VM Created

3.0.6 Step 6 – VM Running

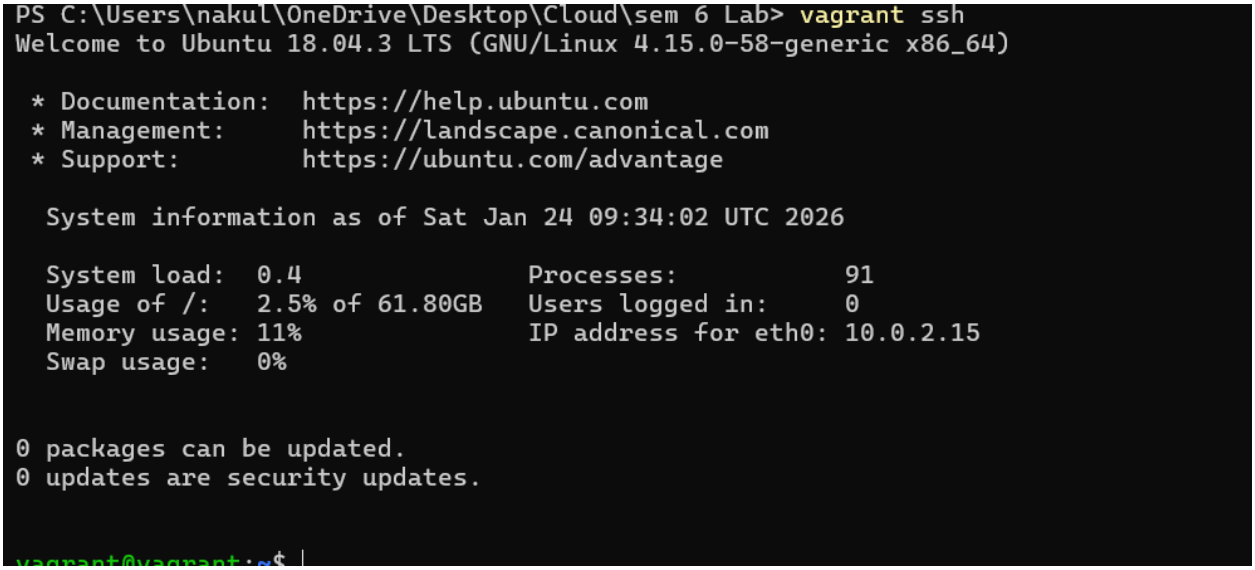


Figure 6: Step6

3.0.7 Step 7 – SSH into VM

```
vagrant ssh
```

3.0.8 Step 8 – Start Nginx in VM

```
sudo systemctl start nginx
```

3.0.9 Step 9 – Verify Nginx

```
curl localhost
```

3.0.10 Step 10 – Virtualization Enabled

---

4 Part B – Docker Container Setup

4.0.1 Step 1 – Open WSL

4.0.2 Step 2 – Run Docker Nginx

```
docker run -d -p 8080:80 --name nginx-container nginx
```

```
vagrant@vagrant:~$ sudo systemctl start nginx
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
vagrant@vagrant:~$
```

Figure 7: Step7

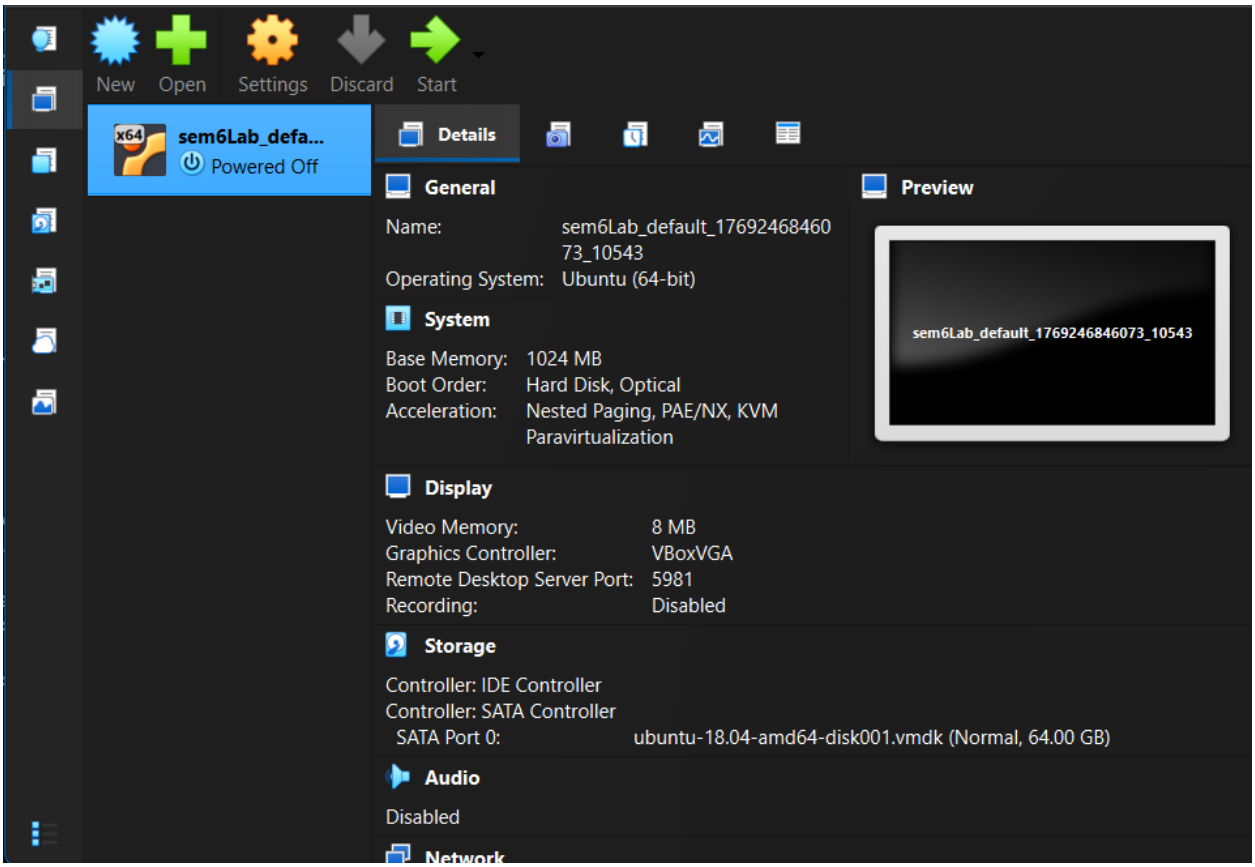


Figure 8: Step8

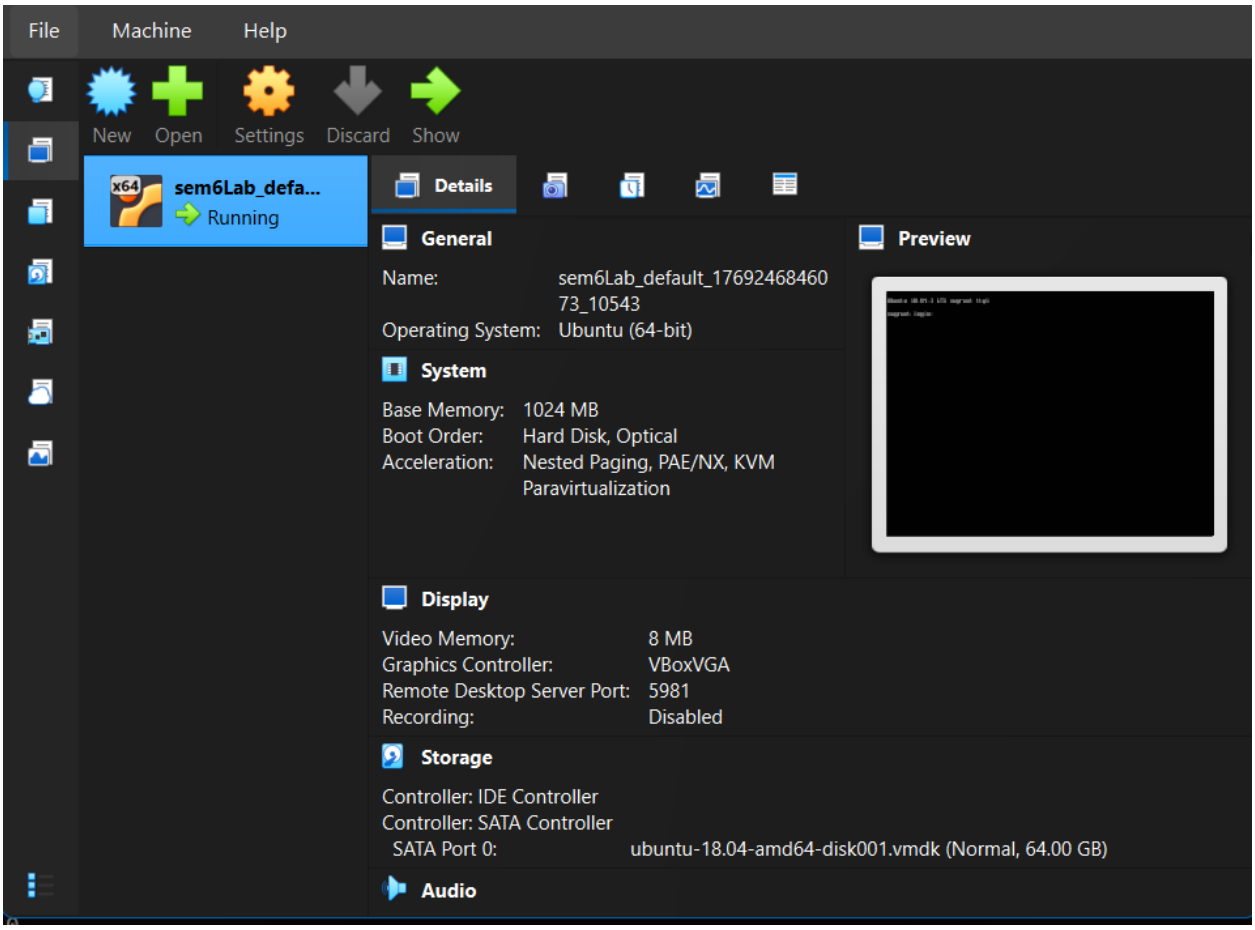


Figure 9: Step9

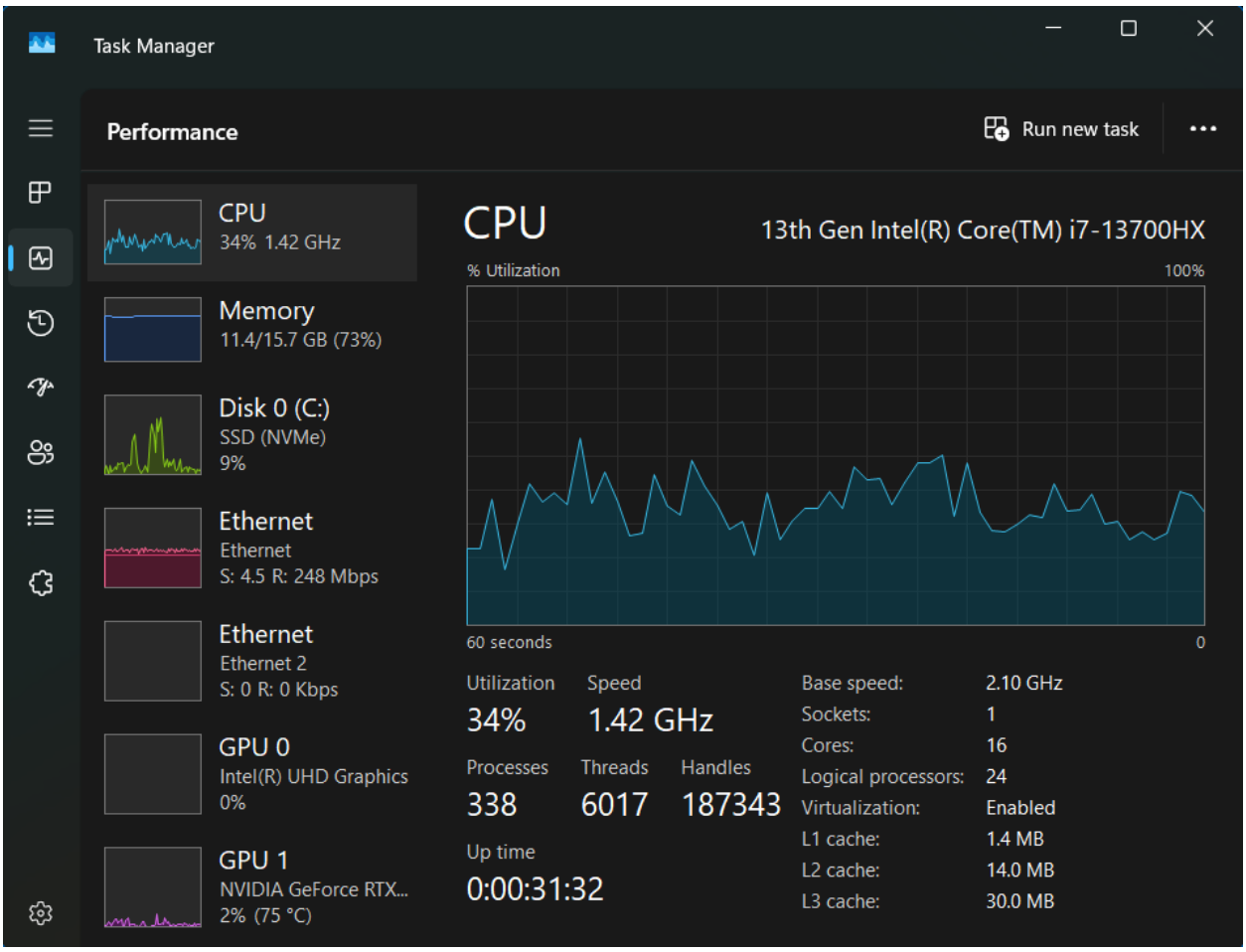


Figure 10: Step10

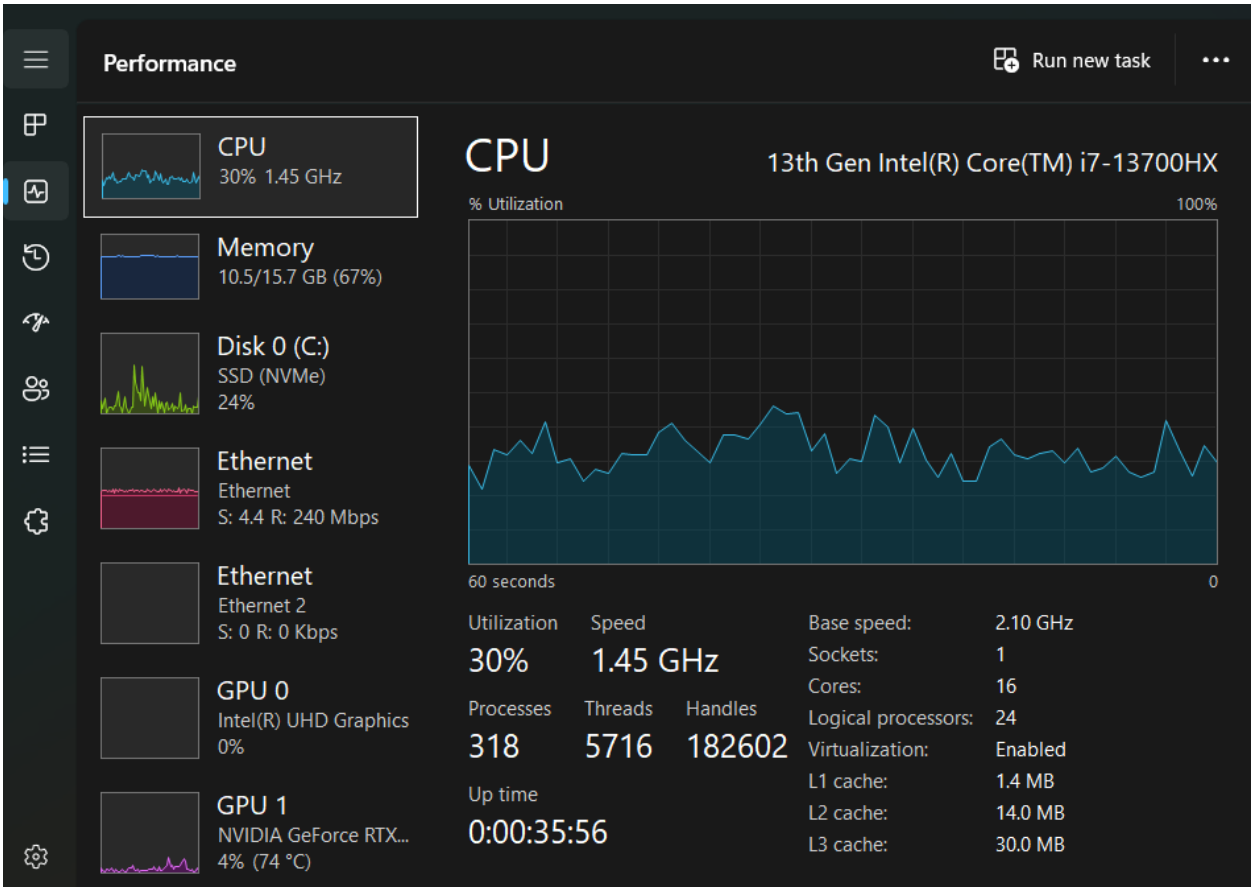
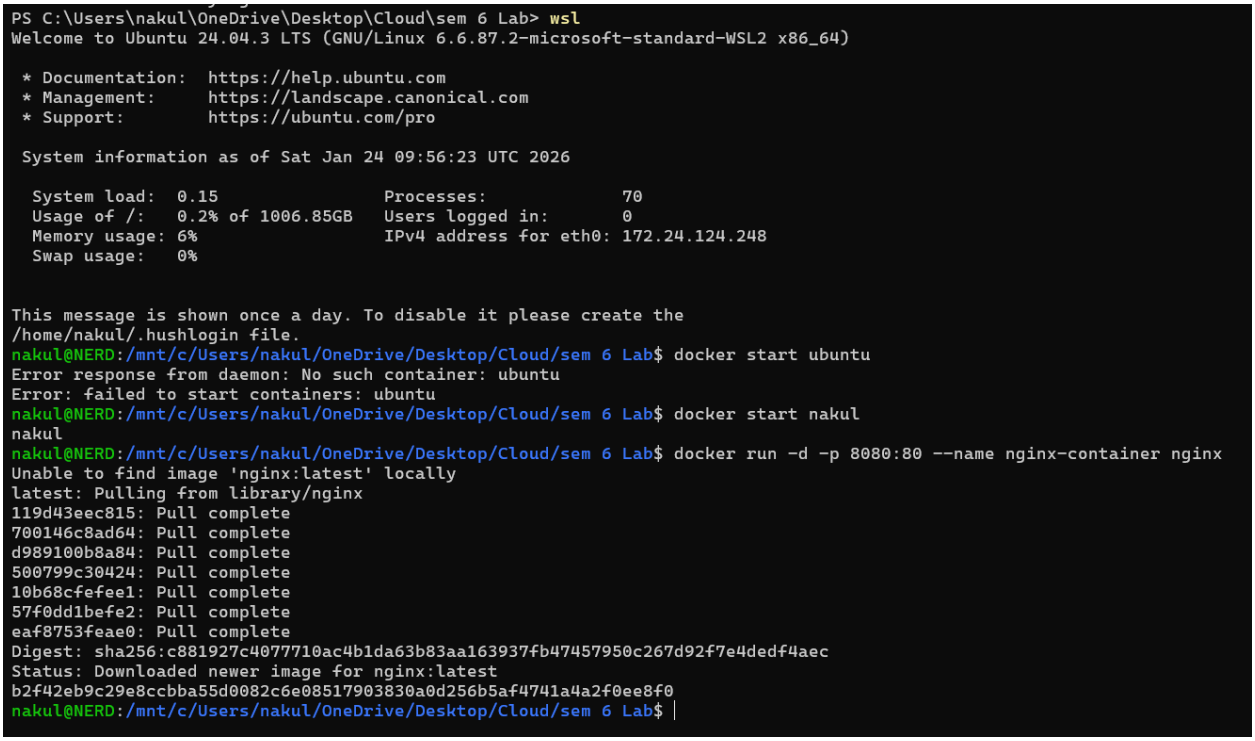


Figure 11: Step11



```
PS C:\Users\nakul\OneDrive\Desktop\Cloud\sem 6 Lab> wsl
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Jan 24 09:56:23 UTC 2026

  System load:  0.15                Processes:             70
  Usage of /:   0.2% of 1006.85GB   Users logged in:       0
  Memory usage: 6%                  IPv4 address for eth0: 172.24.124.248
  Swap usage:   0%


This message is shown once a day. To disable it please create the
/home/nakul/.hushlogin file.
nakul@NERD:/mnt/c/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab$ docker start ubuntu
Error response from daemon: No such container: ubuntu
Error: failed to start containers: ubuntu
nakul@NERD:/mnt/c/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab$ docker start nakul
nakul
nakul@NERD:/mnt/c/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab$ docker run -d -p 8080:80 --name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
b2f42eb9c29e8ccbba55d0082c6e08517903830a0d256b5af4741a4a2f0ee8f0
nakul@NERD:/mnt/c/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab$
```

Figure 12: Step12

4.0.3 Step 3 – Verify Container Output

```
curl localhost:8080
```

4.0.4 Step 4 – Monitor using htop

```
htop
```

4.0.5 Step 5 – Check Memory

```
free -h
```

4.0.6 Step 6 – System Analyze

```
systemd-analyze
```

4.0.7 Step 7 – Docker Stats

```
docker stats
```

4.1 Result

Successfully compared VM and Container performance.

```
nakul@NERD:/mnt/c/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
nakul@NERD:/mnt/c/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab$
```

Figure 13: Step13

```
  0[|          0.7%]   3[           0.0%]   6[           0.0%]  9[           0.0%]  12[          0.0%]  15[          0.0%]  18[          0.0%]  21[          0.0%]
  1[|          1.3%]   4[           0.0%]   7[           0.0%] 10[           0.0%]  13[          0.0%]  16[          0.0%]  19[|         0.7%]  22[          0.0%]
  2[           0.0%]   5[           0.0%]   8[|          0.7%] 11[           0.0%]  14[          0.0%]  17[          0.0%]  20[          0.0%]  23[          0.0%]
Mem[||||||
```

```
nakul@NERD:/mnt/c/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab$ systemd-analyze
Startup finished in 7.326s (userspace)
graphical.target reached after 7.299s in userspace.
nakul@NERD:/mnt/c/Users/nakul/OneDrive/Desktop/Cloud/sem 6 Lab$
```

Figure 16: Step16

```
CONTAINER ID   NAME              CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O         PIDS
b2f42eb9c29e   nginx-container   0.00%     18.79MiB / 7.615GiB   0.24%     1.8kB / 1.4kB   0B / 12.3kB       25
4aed2e87584e   mysql             1.17%     381.9MiB / 7.615GiB   4.90%     1.7kB / 126B    77.4MB / 17.8MB   37
```

Figure 17: Step17

4.2 Navigation

Previous	Home	Next
—	Main README	Lab 2 →

4.2.1 All Labs

- **Lab 1 — VM vs Container Comparison** *(You are here)*
- Lab 2 — Docker Installation & GitHub Pages
- Lab 3 — Docker Images, NGINX & Flask Deployment
- Lab 4 — Docker Essentials
- Lab 5 — Volumes, Env Vars, Monitoring, Networks
- Lab 6 — Docker Run vs Docker Compose

5 Installing Docker on Windows & Making GitHub.io Live

This project documents the complete setup process for:

- Installing and configuring Docker on Windows
- Running containers and testing the environment
- Hosting a static website using GitHub Pages

It serves as a beginner-friendly guide to containerization and basic web deployment.

---

6 Part 1: Installing Docker on Windows

Docker allows applications to run inside lightweight, portable containers.

6.1 Step 1: Install Docker Desktop for Windows

1. Visit the official Docker website:
   https://www.docker.com/products/docker-desktop
2. Download **Docker Desktop for Windows**
3. Run the installer (`Docker Desktop Installer.exe`)
4. During installation:
   - Enable **WSL 2** if prompted
   - Keep default settings
5. Restart your system if required
6. Open Docker Desktop from the Start Menu
7. Wait until it shows **Docker is running**

6.2 Step 2: Verify Docker Installation

Open **Command Prompt** or **PowerShell** and run:

```
docker --version
```

Then run a test container:

```
docker run hello-world
```

If Docker is installed correctly, a confirmation message will be displayed.

6.3 Step 3: Run an Ubuntu Container

```
docker run -it ubuntu:22.04 bash
```

This starts an interactive Ubuntu terminal inside a Docker container where you can execute Linux commands.

6.4 Step 4: Install Java Inside the Container (Example)

Inside the Ubuntu container:

```
apt update
apt install default-jdk -y
java -version
```

Create a Java file:

```
nano Hello.java
```

Add the following code:

```java
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello from Docker on Windows!");
    }
}
```

Compile and run:

```
javac Hello.java
java Hello
```

Expected output:

```
Hello from Docker on Windows!
```

---

7 Part 2: Hosting a Website with GitHub Pages

GitHub Pages allows you to host static websites directly from a GitHub repository.

7.1 Step 1: Create a Repository

1. Go to https://github.com
2. Click **New Repository**
3. Name the repository exactly:

```
yourusername.github.io
```

Replace `yourusername` with your actual GitHub username.

The repository name must match your GitHub username exactly for the site to work.

7.2 Step 2: Add Website Files

Upload your website files such as:

- `index.html`
- `style.css`
- `script.js`

Or push your project using Git from your local machine:

```bash
git init
git add .
git commit -m "Initial website"
```

```
git branch -M main
git remote add origin https://github.com/yourusername/yourusername.github.io.git
git push -u origin main
```

7.3 Step 3: Enable GitHub Pages

1. Open your repository on GitHub
2. Click on **Settings**
3. Click **Pages** from the left sidebar
4. Under Branch, select:

```
Branch: main
Folder: /root
```

5. Click **Save**

7.4 Step 4: Access the Live Website

After a few minutes, your website will be live at:

```
https://yourusername.github.io
```

If the site does not appear immediately, wait a few minutes and refresh the page.

7.5 Screenshot (Example)

7.6 Navigation

Previous	Home	Next
← Lab 1	Main README	Lab 3 →

7.6.1 All Labs

- Lab 1 — VM vs Container Comparison
- **Lab 2 — Docker Installation & GitHub Pages** *(You are here)*
- Lab 3 — Docker Images, NGINX & Flask Deployment
- Lab 4 — Docker Essentials
- Lab 5 — Volumes, Env Vars, Monitoring, Networks

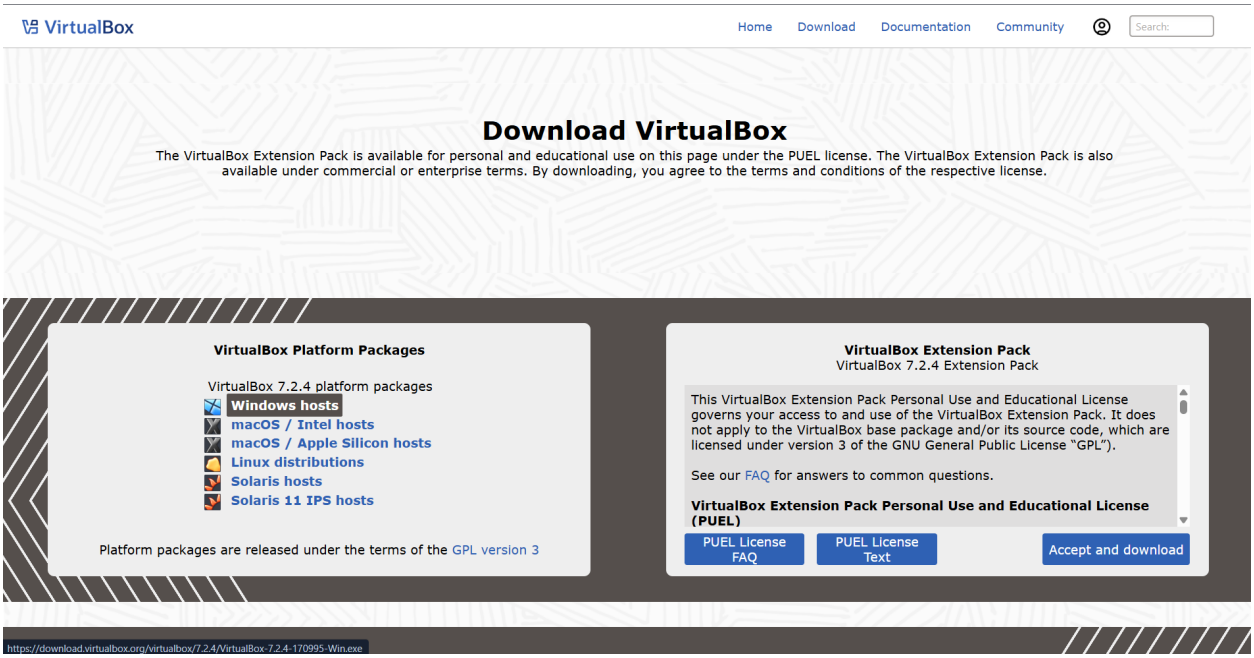


Figure 18: My site live

- Lab 6 — Docker Run vs Docker Compose

8 Lab 3 – Docker Images, NGINX & Flask Application Deployment

8.1 Objective

The objective of this lab was to understand Docker images, container creation, image customization, and deployment of web applications using Docker.

This lab focused on:

- Running official Docker images
- Building custom Docker images
- Comparing image sizes
- Deploying custom HTML using bind mounts
- Containerizing and deploying a Flask web application

---

9 Detailed Implementation Files

For complete step-by-step screenshots and commands, refer to:

 NGINX & Image Customization (Part 1):
View Part 1 README

 Flask Application Containerization (Part 2):
View Flask Container README

---

10 Part A – Working with Official NGINX Image

In this section, we explored how Docker Hub provides ready-to-use images.

10.0.1 What We Did

- Pulled the official NGINX image from Docker Hub
- Ran the container with port mapping
- Verified the running web server in the browser
- Checked image size using docker images

10.0.2 Learning Outcome

- Understood how Docker pulls images from Docker Hub
- Learned how port mapping works (-p host:container)
- Observed how containers isolate services
- Understood basic Docker container lifecycle

---

11 Part B – Custom NGINX Image (Ubuntu Based)

In this section, we created our own NGINX image using Ubuntu as the base image.

11.0.1 What We Did

- Wrote a custom Dockerfile using Ubuntu
- Installed NGINX inside the container
- Built the image using docker build
- Checked and compared image size

11.0.2 Learning Outcome

- Understood how Dockerfiles work
- Learned about base images
- Observed that Ubuntu-based images are larger in size
- Learned image layering concept

---

12 Part C – Lightweight NGINX (Alpine Based)

To optimize image size, we built another image using Alpine Linux.

12.0.1 What We Did

- Used alpine:latest as base image
- Installed NGINX using apk package manager
- Built and ran the container
- Compared image sizes with Ubuntu and official NGINX

12.0.2 Learning Outcome

- Learned why Alpine is preferred for lightweight containers
- Understood image size optimization
- Compared performance and size differences

---

13 Part D – Deploying Custom HTML using Bind Mount

This section demonstrated how Docker can serve local files.

13.0.1 What We Did

- Created a local html directory
- Added a custom index.html file
- Used bind mount (-v) to connect host folder to container
- Verified changes in browser

13.0.2 Learning Outcome

- Understood volume mounting
- Learned difference between image data and runtime data
- Observed real-time file reflection inside container

---

14 Part E – Containerizing a Flask Web Application

In this final section, we deployed a Python Flask web application using Docker.

14.0.1 What We Did

- Created a simple Flask app
- Defined dependencies in requirements.txt

- Wrote a Dockerfile using python:3.9-slim
- Built the Docker image
- Ran the container with port mapping (8080 → 5000)
- Verified the application in browser

14.0.2 Learning Outcome

- Understood how to containerize backend applications
- Learned dependency management in Docker
- Observed how Docker ensures environment consistency
- Understood the concept of the “It works on my machine” problem

---

15 Overall Concepts Covered

- Docker Images
- Containers
- Dockerfile Instructions
- Base Images
- Image Layers
- Port Mapping
- Bind Mounts
- Image Size Optimization
- Application Containerization

---

16 Result

The lab successfully demonstrated:

- Deployment of official Docker images
- Building custom images
- Lightweight container optimization
- Serving custom HTML
- Containerizing and deploying a Flask web application

All applications were successfully accessed through the browser using mapped ports.

---

17 Conclusion

Lab 3 provided practical exposure to real-world Docker usage.

We learned how Docker simplifies application deployment by packaging code, dependencies, and runtime into a portable container. This ensures consistency across development, testing, and production environments.

The lab strengthened understanding of container-based architecture, efficient image building, application deployment strategies, and environment consistency using Docker.

17.1 Navigation

Previous	Home	Next
← Lab 2	Main README	Lab 4 →

17.1.1 All Labs

- Lab 1 — VM vs Container Comparison
- Lab 2 — Docker Installation & GitHub Pages
- **Lab 3 — Docker Images, NGINX & Flask Deployment** *(You are here)*
- Lab 4 — Docker Essentials
- Lab 5 — Volumes, Env Vars, Monitoring, Networks
- Lab 6 — Docker Run vs Docker Compose

18 Lab 4: Docker Essentials

18.1 Objective

Learn Docker fundamentals including containerizing applications with Dockerfiles, building and tagging images, running containers, multi-stage builds, and working with multiple application stacks (Python Flask & Node.js).

18.2 Part 1: Containerizing Applications with Dockerfile

18.2.1 Step 1: Create a Simple Flask Application

app.py — A simple Flask web server with two endpoints:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"
```

```python
@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt:

```
Flask==2.3.3
```

18.2.2 Step 2: Create Dockerfile

```dockerfile
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

18.3 Part 2: Using .dockerignore

18.3.1 .dockerignore File

```
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
```

```
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
```

18.3.2 Why .dockerignore is Important

- **Prevents unnecessary files** from being copied into the image
- **Reduces image size** by excluding development/test files
- **Improves build speed** by reducing build context
- **Increases security** by keeping sensitive files out of images

---

18.4 Part 3: Building Docker Images

18.4.1 Step 1: Basic Build

```
docker build -t my-flask-app .
```

18.4.2 Step 2: Tagging Images

```
docker build -t my-flask-app:1.0 .
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
docker tag my-flask-app:latest my-flask-app:v1.0
```

18.4.3 Step 3: View Image Details

```
docker images
docker history my-flask-app
```

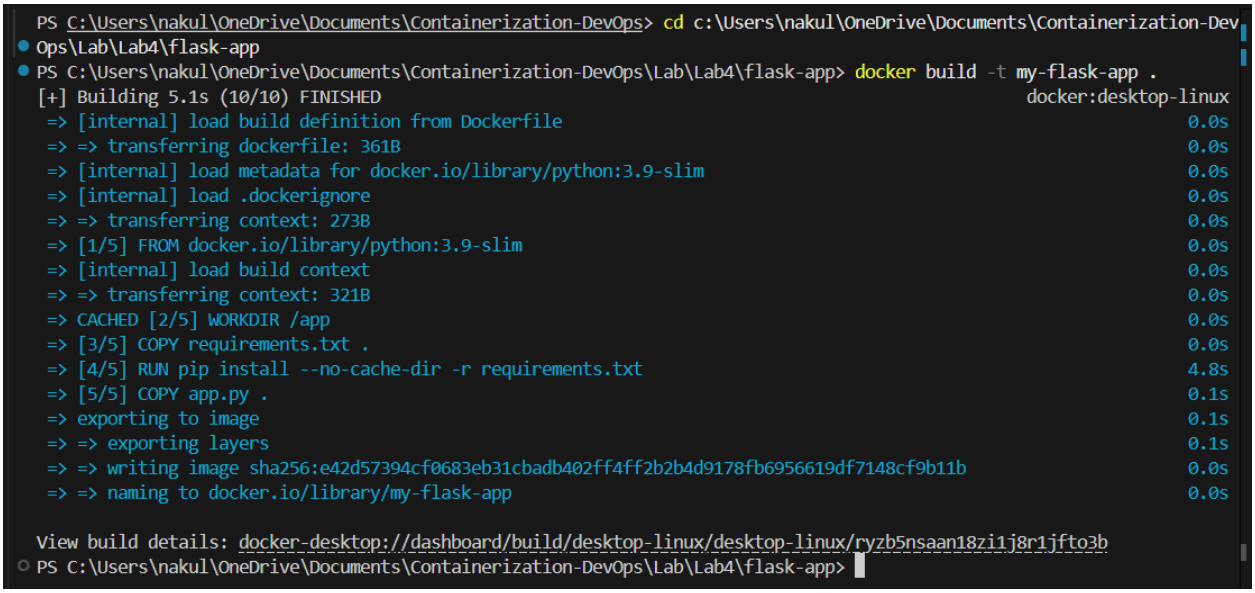


Figure 19: Building Flask Docker Image

```
docker inspect my-flask-app
```

18.5 Part 4: Running Containers

18.5.1 Step 1: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

18.5.2 Step 2: Test & Manage Container

```
curl http://localhost:5000
curl http://localhost:5000/health
docker ps
docker logs flask-container
```

18.5.3 Step 3: Stop & Remove Container

```
docker stop flask-container
docker rm flask-container
```

18.6 Part 5: Multi-stage Builds

18.6.1 Why Multi-stage Builds?

- **Smaller final image size** — only runtime dependencies included

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 63B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                      0.0s
 => exporting to image                                                                                              0.0s
 => => exporting layers                                                                                             0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                        0.0s
 => => naming to docker.io/library/my-flask-app:1.0                                                                 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cvnruc9vjk8gembw1bx5isnan
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:latest -t my-flask-
app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 63B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                      0.0s
 => exporting to image                                                                                              0.0s
 => => exporting layers                                                                                             0.0s
 => => writing image                                                                                                0.0s
 => => naming to doc   docker-desktop://dashboard/build/desktop-linux/desktop-                                     0.0s
 => => naming to doc   linux/x6thk2vxzkbqba8dqivqetw0v (ctrl + click)                                              0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/x6thk2vxzkbqba8dqivqetw0v
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker tag my-flask-app:latest my-flask-app:v1.0
```

Figure 20: Tagging Docker Images

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
assignment1-backend:latest     696e1abfeb2b        185MB             0B
assignment1-db:latest          7068f884d33f        276MB             0B
experiment4:1.0                0e08867af4d9        128MB             0B
experiment4:latest             4231c3a2811e        132MB             0B    U
experiment:1.0                 0e08867af4d9        128MB             0B
flask-multistage:latest        94a39ca50667        145MB             0B
flask-regular:latest           4231c3a2811e        132MB             0B    U
my-flask-app:1.0               e42d57394cf0        132MB             0B
my-flask-app:latest            e42d57394cf0        132MB             0B
my-flask-app:v1.0              e42d57394cf0        132MB             0B
mysql:8                        a0f6c7786c75        777MB             0B    U
mysql:8.0                      6f19538dd7d2        780MB             0B
nakul71/my-flask-app:1.0       0e08867af4d9        128MB             0B
nginx-alpine:latest            a33d5a30a4a0       10.4MB             0B
nginx-ubuntu:latest            5363ea9a780d        207MB             0B
nginx:latest                   5cdef4ac3335        161MB             0B
python:3.9-slim                085da638e1b8        122MB             0B
ubuntu:latest                  493218ed0f40       78.1MB             0B
wordpress:latest               6fe8e9457b10        752MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker history my-flask-app
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e42d57394cf0   11 minutes ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY app.py . # buildkit                        228B      buildkit.dockerfile.v0
<missing>      11 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   10.5MB    buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY requirements.txt . # buildkit              13B       buildkit.dockerfile.v0
<missing>      3 weeks ago      WORKDIR /app                                    0B        buildkit.dockerfile.v0
<missing>      4 months ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  for src in idle3 p…   36B       buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…   39.5MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  apt-get update;  a…   3.81MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   78.6MB    debuerreotype 0.16
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
>> docker inspect my-flask-app
[
    {
        "Id": "sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b",
        "RepoTags": [
            "my-flask-app:1.0",
            "my-flask-app:latest",
            "my-flask-app:v1.0"
        ],
```

Figure 21: Docker Images, History, and Inspect

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker run -d -p 5000:5000 --name flask-containe
r my-flask-app
>>
6f0107357708434 1c00e87d2de6ff354c17dc102264f25649799bc31c2a11c49
```

Figure 22: Running Flask Container

```
inerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:21 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : OK
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 2
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 20 Mar 2026 04:32:31 GMT
                    Server: Werkzeug/3.1.6 Python/3.9.25

                    OK
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:31 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED         STATUS         PORTS                                       NAMES
6f0107357708   my-flask-app   "python app.py"   7 minutes ago   Up 7 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

Figure 23: Testing Flask Endpoints and Container Status

- **Better security** — build tools not present in final image
- **Separation of concerns** — build stage vs runtime stage

18.6.2 Multi-stage Dockerfile (`Dockerfile.multistage`)

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder
WORKDIR /app

COPY requirements.txt .
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim
WORKDIR /app

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY app.py .

RUN useradd -m -u 1000 appuser
USER appuser

EXPOSE 5000
CMD ["python", "app.py"]
```

18.6.3 Build & Compare

```
docker build -t flask-regular .
docker build -f Dockerfile.multistage -t flask-multistage .
docker images | findstr flask-
```

Results: | Image | Size | |——-|——| | flask-regular | 132MB | | flask-multistage | 145MB |

> **Note:** The multi-stage build includes a virtual environment and a non-root user for better security practices, which adds slight overhead. The benefit of multi-stage becomes more significant with larger applications that have heavy build dependencies.

18.7 Part 6: Publishing to Docker Hub

```
docker login
docker tag my-flask-app:latest username/my-flask-app:1.0
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t flask-regular .
 => [internal] load .dockerignore                                                                                    0.0s
 => => transferring context: 273B                                                                                    0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                     0.0s
 => [internal] load build context                                                                                    0.0s
 => => transferring context: 63B                                                                                     0.0s
 => CACHED [2/5] WORKDIR /app                                                                                        0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                             0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                  0.0s
 => CACHED [5/5] COPY app.py .                                                                                       0.0s
 => exporting to image                                                                                               0.0s
 => => exporting layers                                                                                              0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                         0.0s
 => => naming to docker.io/library/flask-regular                                                                     0.0s

 View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hcwinfxyz9znl4zzathqwj838
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -f Dockerfile.multistage -t flask-multistage .
>>
[+] Building 8.2s (13/13) FINISHED                                                                   docker:desktop-linux
 => [internal] load build definition from Dockerfile.multistage                                                      0.0s
 => => transferring dockerfile: 486B                                                                                 0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                   0.0s
 => [internal] load .dockerignore                                                                                    0.0s
 => => transferring context: 273B                                                                                    0.0s
 => [builder 1/5] FROM docker.io/library/python:3.9-slim                                                             0.0s
 => [internal] load build context                                                                                    0.0s
 => => transferring context: 63B                                                                                     0.0s
 => CACHED [builder 2/5] WORKDIR /app                                                                                0.0s
 => CACHED [builder 3/5] COPY requirements.txt .                                                                     0.0s
 => [builder 4/5] RUN python -m venv /opt/venv                                                                       3.4s
 => [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt                                                 3.3s
 => [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv                                                            0.2s
 => [stage-1 4/5] COPY app.py .                                                                                      0.0s
 => [stage-1 5/5] RUN useradd -m -u 1000 appuser                                                                     0.2s
 => exporting to image                                                                                               0.2s
 => => exporting layers                                                                                              0.2s
 => => writing image sha256:15e019e88dc649d3371adfd7203641fbae971a4cb776c52bb56e64a89bde403a                         0.0s
 => => naming to docker.io/library/flask-multistage                                                                  0.0s

 View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hm4w1vg4rgjqp4u2ou41tekcr
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images | findstr flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      15e019e88dc6        145MB             0B
flask-regular:latest         e42d57394cf0        132MB             0B
my-flask-app:1.0             e42d57394cf0        132MB             0B
my-flask-app:latest          e42d57394cf0        132MB             0B
my-flask-app:v1.0            e42d57394cf0        132MB             0B
nakul71/my-flask-app:1.0     0e08867af4d9        128MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 24: Multi-stage Build and Size Comparison

```
docker tag my-flask-app:latest username/my-flask-app:latest
docker push username/my-flask-app:1.0
docker push username/my-flask-app:latest
docker pull username/my-flask-app:latest
docker run -d -p 5000:5000 username/my-flask-app:latest
```

18.8 Part 7: Node.js Example

18.8.1 Application Files

app.js:

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

package.json:

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

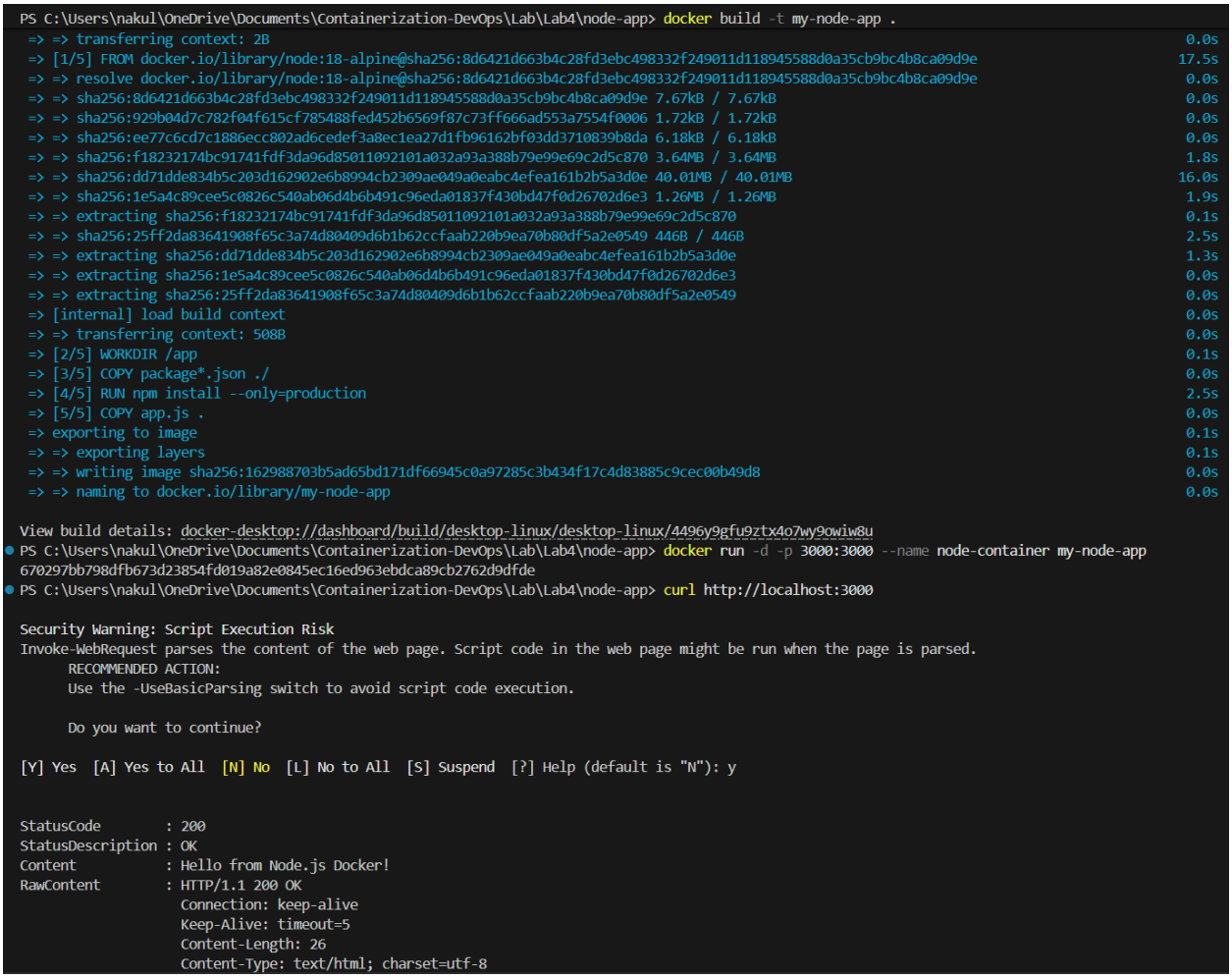
Dockerfile:

```
FROM node:18-alpine
WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .
```

```
EXPOSE 3000
CMD ["node", "app.js"]
```

18.8.2 Build & Run

```
docker build -t my-node-app .
docker run -d -p 3000:3000 --name node-container my-node-app
curl http://localhost:3000
```



```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker build -t my-node-app .
 => => transferring context: 2B                                                                                    0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  17.5s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  0.0s
 => => sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 7.67kB / 7.67kB                     0.0s
 => => sha256:929b04d7c782f04f615cf785488fed452b6569f87c73ff666ad553a7554f0006 1.72kB / 1.72kB                     0.0s
 => => sha256:ee77c6cd7c1886ecc802ad6cedef3a8ec1ea27d1fb96162bf03dd3710839b8da 6.18kB / 6.18kB                     0.0s
 => => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB                     1.8s
 => => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB                   16.0s
 => => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB                     1.9s
 => => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870                          0.1s
 => => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B                         2.5s
 => => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e                          1.3s
 => => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3                          0.0s
 => => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549                          0.0s
 => [internal] load build context                                                                                  0.0s
 => => transferring context: 508B                                                                                  0.0s
 => [2/5] WORKDIR /app                                                                                             0.1s
 => [3/5] COPY package*.json ./                                                                                    0.0s
 => [4/5] RUN npm install --only=production                                                                        2.5s
 => [5/5] COPY app.js .                                                                                            0.0s
 => exporting to image                                                                                             0.1s
 => => exporting layers                                                                                            0.1s
 => => writing image sha256:162988703b5ad65bd171df66945c0a97285c3b434f17c4d83885c9cec00b49d8                        0.0s
 => => naming to docker.io/library/my-node-app                                                                     0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/4496y9gfu9ztx4o7wy9owiw8u
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker run -d -p 3000:3000 --name node-container my-node-app
670297bb798dfb673d23854fd019a82e0845ec16ed963ebdca89cb2762d9dfde
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> curl http://localhost:3000

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : Hello from Node.js Docker!
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Keep-Alive: timeout=5
                    Content-Length: 26
                    Content-Type: text/html; charset=utf-8
```

Figure 25: Node.js Docker Build, Run, and Curl

18.8.3 Container Status & Logs

```
docker ps
docker logs node-container
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS         PORTS                                       NAMES
670297bb798d   my-node-app   "docker-entrypoint.s…"   47 seconds ago   Up 47 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  node-container
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker logs node-container
Server running on port 3000
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app>
```

Figure 26: Node.js Container Status and Logs

18.9 Part 8: Practice Exercises

18.9.1 Exercise 1: Multi-tagging

```
docker build -t myapp:latest -t myapp:v2.0 .
docker images | findstr myapp
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t myapp:latest -t myapp:v2.0 .
[+] Building 0.1s (10/10) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 361B                                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                                  0.0s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 273B                                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                                    0.0s
 => [internal] load build context                                                                                                   0.0s
 => => transferring context: 63B                                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                                      0.0s
 => exporting to image                                                                                                              0.0s
 => => exporting layers                                                                                                             0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                                        0.0s
 => => naming to docker.io/library/myapp:latest                                                                                     0.0s
 => => naming to docker.io/library/myapp:v2.0                                                                                       0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/kn78zmmhr53ius6sb0v6n75fo
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images | findstr myapp
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
myapp:latest                e42d57394cf0        132MB             0B
myapp:v2.0                  e42d57394cf0        132MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 27: Multi-tagging Exercise

18.9.2 Exercise 3: Clean Build vs Cached Build

```
docker build --no-cache -t clean-app .     # ~5.1s (no cache)
docker build -t cached-app .               # ~0.1s (cached)
```

Results: | Build Type | Time | Cache Used | |———-|——|————| | Clean (no-cache) | 5.1s | □ No | | Cached | 0.1s | □ Yes |

The cached build is significantly faster as Docker reuses unchanged layers.

18.10 Cleanup

```
docker stop node-container
docker rm node-container
docker container prune
docker image prune
docker system prune -a
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build --no-cache -t clean-app .
[+] Building 5.1s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 361B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                    0.0s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 273B                                                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                      0.0s
 => [internal] load build context                                                                                     0.0s
 => => transferring context: 63B                                                                                      0.0s
 => CACHED [2/5] WORKDIR /app                                                                                         0.0s
 => [3/5] COPY requirements.txt .                                                                                     0.0s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                          6.8s
 => [5/5] COPY app.py .                                                                                               0.1s
 => exporting to image                                                                                                0.1s
 => => exporting layers                                                                                               0.1s
 => => writing image sha256:15d903280f371b859fafaae680bb3337e02f86e1ba82a72e110acebecadb81a0                          0.0s
 => => naming to docker.io/library/clean-app                                                                          0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/rd6x0l50xpyx2l9gd3m983deq
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t cached-app .
[+] Building 0.1s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 361B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                    0.0s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 273B                                                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                      0.0s
 => [internal] load build context                                                                                     0.0s
 => => transferring context: 63B                                                                                      0.0s
 => CACHED [2/5] WORKDIR /app                                                                                         0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                              0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                   0.0s
 => CACHED [5/5] COPY app.py .                                                                                        0.0s
 => exporting to image                                                                                                0.0s
 => => exporting layers                                                                                               0.0s
 => => writing image sha256:15d903280f371b859fafaae680bb3337e02f86e1ba82a72e110acebecadb81a0                          0.0s
 => => naming to docker.io/library/cached-app                                                                         0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/amzemwfbrkk8vuiusyxvrrwvn
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 28: Clean Build vs Cached Build

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/amzemwfbrkk8vuiusyxvrrwvn
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker stop node-container
>> docker rm node-container
>> docker container prune
>> docker image prune
>> docker system prune -a
>>
node-container
node-container
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
a715a2e7e98f87a70b987867c4de8596ea6000a443861826f3a18d3dd619c380
bda5a82e26dc24909630bdc433b4200373b84d0f8810f2a40019e1099edd38d2

Total reclaimed space: 118kB
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Deleted Images:
deleted: sha256:94a39ca50667244fc407637f3233bda382d2d7fcdd6c739bc5dca9ea72ac13de

Total reclaimed space: 0B
WARNING! This will remove:
  - all stopped containers
  - all networks not used by at least one container
  - all images without at least one container associated to them
  - all build cache

Are you sure you want to continue? [y/N] y
Deleted Networks:
crud_default
backend_default

Deleted Images:
untagged: nginx-alpine:latest
deleted: sha256:a33d5a30a4a003eac0cb8ca0422adad4e25d3af36be32ead5beb451a49c451c7
untagged: experiment4:1.0
untagged: experiment:1.0
untagged: nakul71/my-flask-app:1.0
deleted: sha256:0e08867af4d91833c639d70579f7c0461e5d6b22d96a6fc993b526d7059ab370
untagged: experiment4:latest
deleted: sha256:4231c3a2811e6ac2bfcbd0efedc703673f3e4d15e5bc446c5ee1882509c8b62c
untagged: my-node-app:latest
deleted: sha256:162988703b5ad65bd171df66945c0a97285c3b434f17c4d83885c9cec00b49d8
```

Figure 29: Docker Cleanup

18.11 Docker Commands Cheat Sheet

Command	Example
Build	`docker build -t myapp .`
Run	`docker run -p 3000:3000 myapp`
List containers	`docker ps -a`
List images	`docker images`
Tag	`docker tag myapp:latest myapp:v1`
Login	`docker login`
Push	`docker push username/myapp`
Pull	`docker pull username/myapp`
Remove container	`docker rm container-name`
Remove image	`docker rmi image-name`
Logs	`docker logs container-name`
Exec	`docker exec -it container-name bash`

18.12 Key Takeaways

1. **Dockerfile** defines how to build a container image step by step
2. **.dockerignore** improves build performance and security
3. **Tagging** enables proper image versioning and organization
4. **Multi-stage builds** separate build and runtime for smaller, more secure images
5. **Docker Hub** enables sharing and distributing images
6. **Always test locally** before pushing to production
7. **Cleanup regularly** using `docker system prune` to reclaim disk space

18.13 □ Navigation

Previous	Home	Next
← Lab 3	Main README	Lab 5 →

18.13.1 All Labs

- Lab 1 — VM vs Container Comparison
- Lab 2 — Docker Installation & GitHub Pages
- Lab 3 — Docker Images, NGINX & Flask Deployment
- **Lab 4 — Docker Essentials** *(You are here)*
- Lab 5 — Volumes, Env Vars, Monitoring, Networks
- Lab 6 — Docker Run vs Docker Compose

19 Lab 5: Docker Volumes, Environment Variables, Monitoring & Networks

19.1 Objective

Learn advanced Docker concepts including persistent data storage with volumes, application configuration with environment variables, container monitoring, and linking multiple containers using Docker networks.

19.2 Part 1: Docker Volumes – Persistent Data Storage

By default, data inside a container is ephemeral (lost when the container is removed). Volumes provide a way to safely persist data.

19.2.1 1. Anonymous & Named Volumes

```
docker run -d -v /app/data --name web1 nginx
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
```

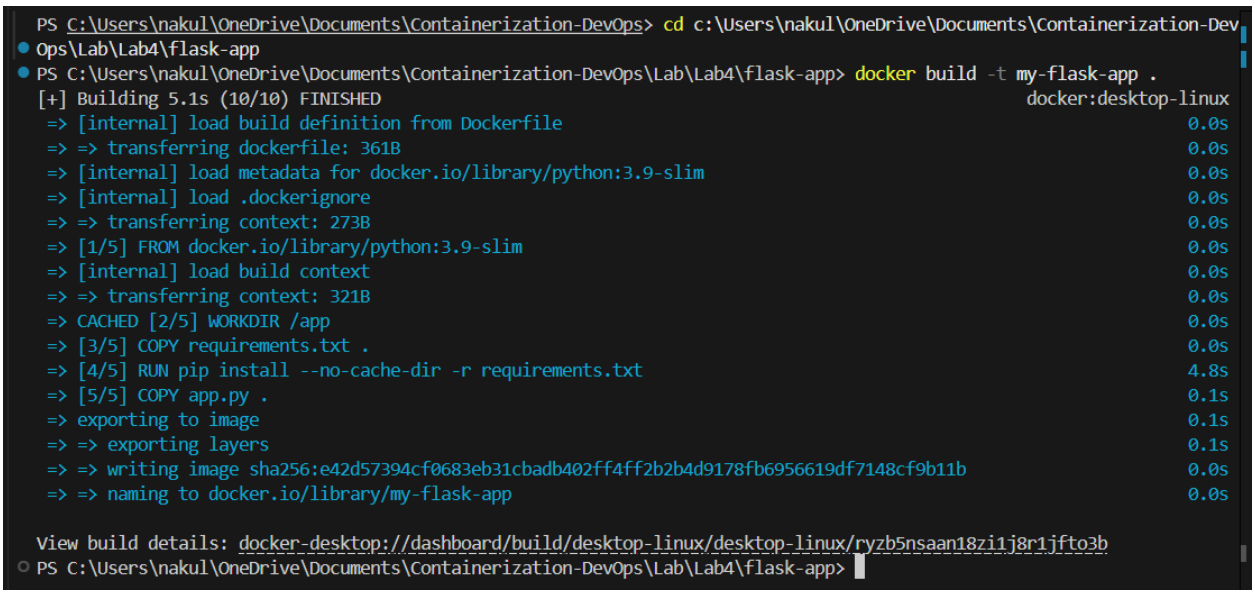


Figure 30: Volumes Creation and List

19.2.2 2. Bind Mounts (Host Directory)

Bind mounts attach a specific directory on your host machine to a directory inside the container.

```
mkdir myapp-data
docker run -d -v ${PWD}/myapp-data:/app/data --name web3 nginx
echo "From Host" > myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 63B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                      0.0s
 => exporting to image                                                                                              0.0s
 => => exporting layers                                                                                             0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                        0.0s
 => => naming to docker.io/library/my-flask-app:1.0                                                                 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cvnruc9vjk8gembw1bx5isnan
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:latest -t my-flask-
app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 63B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                      0.0s
 => exporting to image                                                                                              0.0s
 => => exporting layers                                                                                             0.0s
 => => writing image                                                                                                0.0s
 => => naming to doc   docker-desktop://dashboard/build/desktop-linux/desktop-                                     0.0s
 => => naming to doc   linux/x6thk2vxzkbqba8dqivqetw0v (ctrl + click)                                              0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/x6thk2vxzkbqba8dqivqetw0v
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker tag my-flask-app:latest my-flask-app:v1.0
```

Figure 31: Bind Mount Working

19.3 Part 2: Environment Variables

Environment variables allow us to pass configuration parameters into the container at runtime.

19.3.1 1. Building and Running a Flask App with ENV Vars

We built a custom Flask application that returns configuration data via a `/config` API endpoint.

app.py

```python
import os
from flask import Flask
app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

Dockerfile

```dockerfile
FROM python:3.9-slim

ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000
CMD ["python", "app.py"]
```

```bash
docker build -t flask-env-app .
docker run -d --name flask-app -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e API_KEY="secret-xyz" \
  flask-env-app
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
assignment1-backend:latest     696e1abfeb2b       185MB             0B
assignment1-db:latest          7068f884d33f       276MB             0B
experiment4:1.0                0e08867af4d9       128MB             0B
experiment4:latest             4231c3a2811e       132MB             0B    U
experiment:1.0                 0e08867af4d9       128MB             0B
flask-multistage:latest        94a39ca50667       145MB             0B
flask-regular:latest           4231c3a2811e       132MB             0B    U
my-flask-app:1.0               e42d57394cf0       132MB             0B
my-flask-app:latest            e42d57394cf0       132MB             0B
my-flask-app:v1.0              e42d57394cf0       132MB             0B
mysql:8                        a0f6c7786c75       777MB             0B    U
mysql:8.0                      6f19538dd7d2       780MB             0B
nakul71/my-flask-app:1.0       0e08867af4d9       128MB             0B
nginx-alpine:latest            a33d5a30a4a0      10.4MB             0B
nginx-ubuntu:latest            5363ea9a780d       207MB             0B
nginx:latest                   5cdef4ac3335       161MB             0B
python:3.9-slim                085da638e1b8       122MB             0B
ubuntu:latest                  493218ed0f40      78.1MB             0B
wordpress:latest               6fe8e9457b10       752MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker history my-flask-app
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e42d57394cf0   11 minutes ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY app.py . # buildkit                        228B      buildkit.dockerfile.v0
<missing>      11 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   10.5MB    buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY requirements.txt . # buildkit              13B       buildkit.dockerfile.v0
<missing>      3 weeks ago      WORKDIR /app                                    0B        buildkit.dockerfile.v0
<missing>      4 months ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  for src in idle3 p…   36B       buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…   39.5MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  apt-get update;  a…   3.81MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   78.6MB    debuerreotype 0.16
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
>> docker inspect my-flask-app
[
    {
        "Id": "sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b",
        "RepoTags": [
            "my-flask-app:1.0",
            "my-flask-app:latest",
            "my-flask-app:v1.0"
        ],
```

Figure 32: App Build and Run with ENV

19.3.2 2. Testing Injected Environment Variables

We can verify that the environment variables were successfully passed into the running container.

```
docker exec flask-app printenv DATABASE_HOST
curl http://localhost:5000/config
```

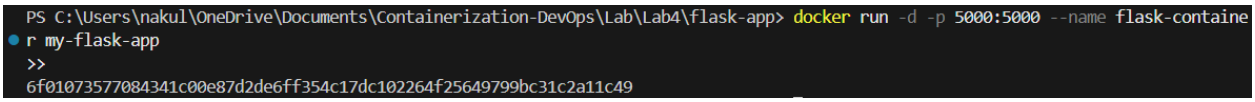


Figure 33: Testing Environment Variables with Curl

19.4 Part 3: Docker Monitoring

Monitoring allows us to keep track of resource usage, running processes, and application logs.

```
docker stats --no-stream
docker top flask-app
docker logs flask-app
```

19.4.1 Automated Monitoring Script

We can automate these checks using a bash script (`monitor.sh`):

monitor.sh

```bash
#!/bin/bash

echo "=== Docker Monitoring Dashboard ==="
echo "Time: $(date)"

echo "1. Running Containers:"
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

echo "2. Resource Usage:"
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"

echo "3. Recent Events:"
docker events --since '5m' --until '0s' | tail

echo "4. System Info:"
docker system df
```

```
inerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:21 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : OK
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 2
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 20 Mar 2026 04:32:31 GMT
                    Server: Werkzeug/3.1.6 Python/3.9.25

                    OK
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:31 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED         STATUS         PORTS                                       NAMES
6f0107357708   my-flask-app   "python app.py"   7 minutes ago   Up 7 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

Figure 34: Docker Stats, Top, and Logs

19.5 Part 4: Docker Networks

Docker networks allow isolated containers to securely communicate with each other.

19.5.1 1. Creating a Network

We created a custom bridge network and attached two Nginx containers to it.

```
docker network create my-network
docker run -d --name net-web1 --network my-network nginx
docker run -d --name net-web2 --network my-network nginx
```

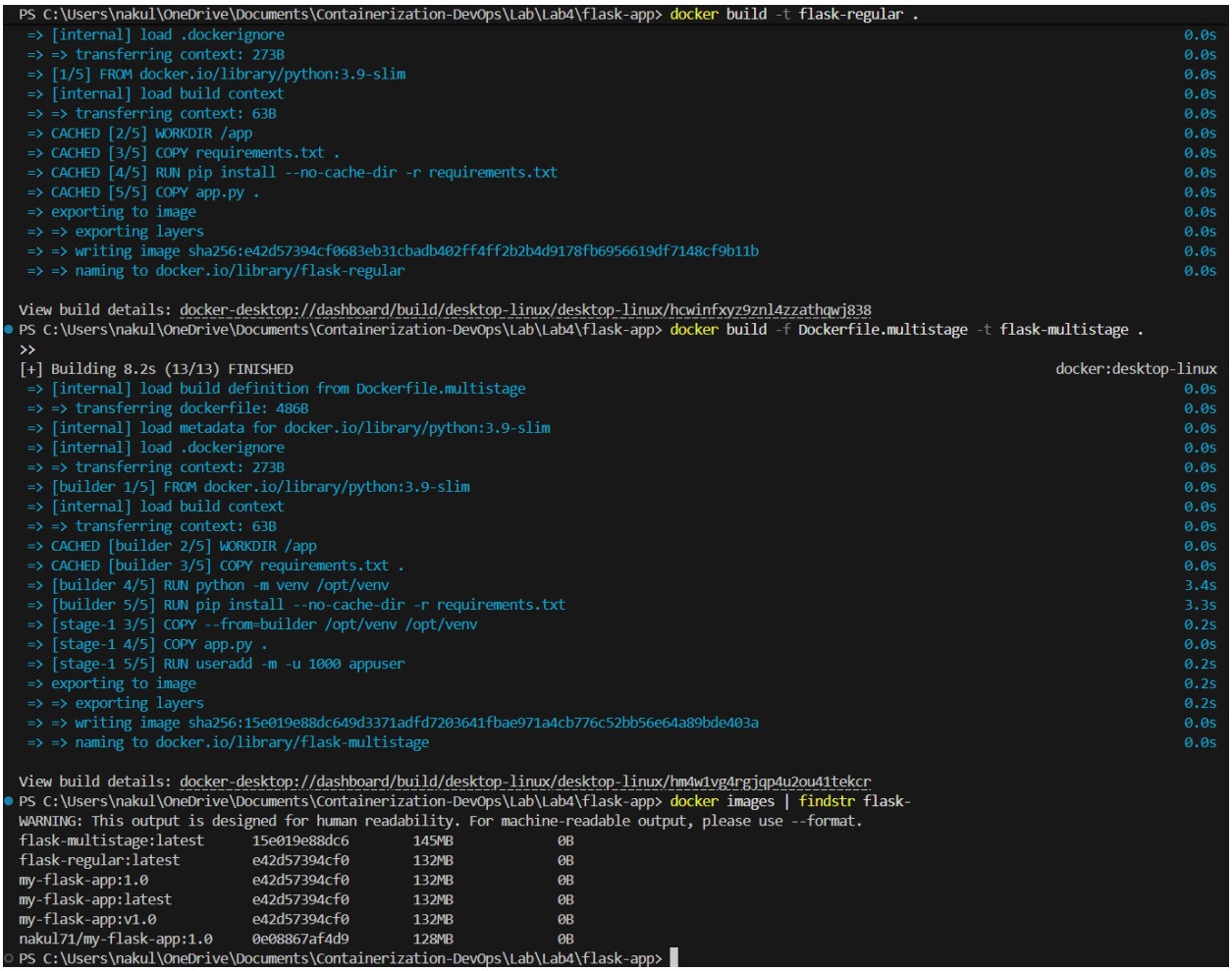


Figure 35: Network Creation and Container Attach

19.5.2 2. Internal Network Communication

Because they share a custom network, containers can communicate with each other using their container names as DNS addresses.

```
docker exec net-web1 curl -s http://net-web2
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker build -t my-node-app .
 => => transferring context: 2B                                                                                    0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  17.5s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  0.0s
 => => sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 7.67kB / 7.67kB                     0.0s
 => => sha256:929b04d7c782f04f615cf785488fed452b6569f87c73ff666ad553a7554f0006 1.72kB / 1.72kB                     0.0s
 => => sha256:ee77c6cd7c1886ecc802ad6cedef3a8ec1ea27d1fb96162bf03dd3710839b8da 6.18kB / 6.18kB                     0.0s
 => => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB                     1.8s
 => => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB                   16.0s
 => => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB                     1.9s
 => => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870                          0.1s
 => => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B                         2.5s
 => => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e                          1.3s
 => => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3                          0.0s
 => => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549                          0.0s
 => [internal] load build context                                                                                  0.0s
 => => transferring context: 508B                                                                                  0.0s
 => [2/5] WORKDIR /app                                                                                             0.1s
 => [3/5] COPY package*.json ./                                                                                    0.0s
 => [4/5] RUN npm install --only=production                                                                        2.5s
 => [5/5] COPY app.js .                                                                                            0.0s
 => exporting to image                                                                                             0.1s
 => => exporting layers                                                                                            0.1s
 => => writing image sha256:162988703b5ad65bd171df66945c0a97285c3b434f17c4d83885c9cec00b49d8                        0.0s
 => => naming to docker.io/library/my-node-app                                                                     0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/4496y9gfu9ztx4o7wy9owiw8u
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker run -d -p 3000:3000 --name node-container my-node-app
670297bb798dfb673d23854fd019a82e0845ec16ed963ebdca89cb2762d9dfde
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> curl http://localhost:3000

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : Hello from Node.js Docker!
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Keep-Alive: timeout=5
                    Content-Length: 26
                    Content-Type: text/html; charset=utf-8
```

Figure 36: Network Internal Communication via Curl

19.6 Part 5: Real-World Example (Multi-Container Setup)

In a real-world scenario, we often have an application connected to a database and a cache layer, all residing on the same network.

19.6.1 1. Running Postgres and Redis on a Shared Network

```
docker network create myapp-network
docker run -d --name postgres --network myapp-network \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -v postgres-data:/var/lib/postgresql/data postgres:15

docker run -d --name redis --network myapp-network \
  -v redis-data:/data redis:7-alpine
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          PORTS                                          NAMES
670297bb798d   my-node-app   "docker-entrypoint.s…"   47 seconds ago   Up 47 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp    node-container
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker logs node-container
Server running on port 3000
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app>
```

Figure 37: Postgres and Redis Network Setup

19.6.2 2. Verifying the Services

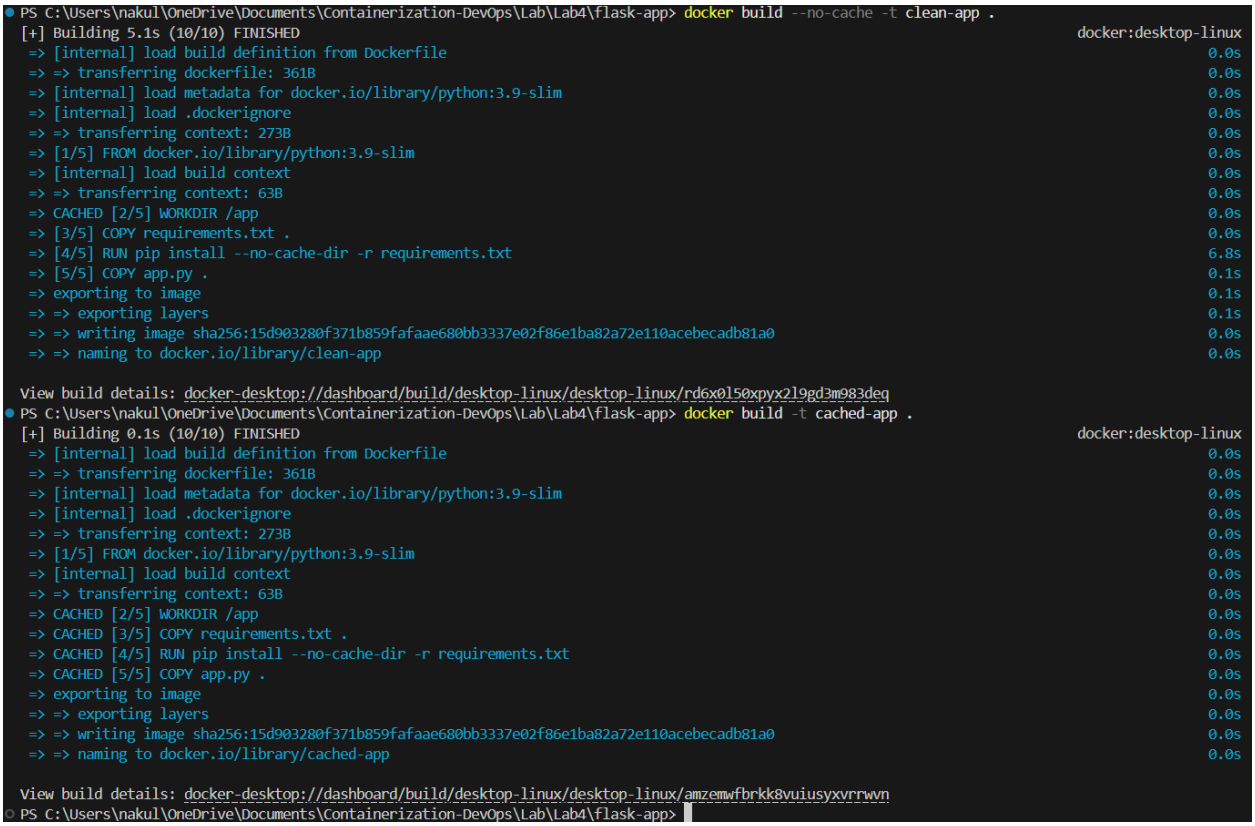
```
docker ps
docker stats --no-stream postgres redis
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t myapp:latest -t myapp:v2.0 .
[+] Building 0.1s (10/10) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 361B                                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                                  0.0s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 273B                                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                                    0.0s
 => [internal] load build context                                                                                                   0.0s
 => => transferring context: 63B                                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                                      0.0s
 => exporting to image                                                                                                              0.0s
 => => exporting layers                                                                                                             0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                                        0.0s
 => => naming to docker.io/library/myapp:latest                                                                                     0.0s
 => => naming to docker.io/library/myapp:v2.0                                                                                       0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/kn78zmmhr53ius6sb0v6n75fo
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images | findstr myapp
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
myapp:latest                e42d57394cf0        132MB             0B
myapp:v2.0                  e42d57394cf0        132MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 38: Verifying Running Services

19.7 Cleanup

```
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)
docker volume prune -f
docker network prune -f
```



```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build --no-cache -t clean-app .
[+] Building 5.1s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 361B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                    0.0s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 273B                                                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                      0.0s
 => [internal] load build context                                                                                     0.0s
 => => transferring context: 63B                                                                                      0.0s
 => CACHED [2/5] WORKDIR /app                                                                                         0.0s
 => [3/5] COPY requirements.txt .                                                                                     0.0s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                          6.8s
 => [5/5] COPY app.py .                                                                                               0.1s
 => exporting to image                                                                                                0.1s
 => => exporting layers                                                                                               0.1s
 => => writing image sha256:15d903280f371b859fafaae680bb3337e02f86e1ba82a72e110acebecadb81a0                          0.0s
 => => naming to docker.io/library/clean-app                                                                          0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/rd6x0l50xpyx2l9gd3m983deq
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t cached-app .
[+] Building 0.1s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 361B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                    0.0s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 273B                                                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                      0.0s
 => [internal] load build context                                                                                     0.0s
 => => transferring context: 63B                                                                                      0.0s
 => CACHED [2/5] WORKDIR /app                                                                                         0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                              0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                   0.0s
 => CACHED [5/5] COPY app.py .                                                                                        0.0s
 => exporting to image                                                                                                0.0s
 => => exporting layers                                                                                               0.0s
 => => writing image sha256:15d903280f371b859fafaae680bb3337e02f86e1ba82a72e110acebecadb81a0                          0.0s
 => => naming to docker.io/library/cached-app                                                                         0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/amzemwfbrkk8vuiusyxvrrwvn
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 39: Docker Cleanup Output

19.8 Docker Commands Cheat Sheet

19.8.1 Volumes

Command	Description
`docker volume create <name>`	Create a named volume
`docker run -v <volume>:/path`	Mount named volume
`docker run -v /host:/container`	Mount local bind

19.8.2 Environment Variables

Command	Description
`docker run -e VAR=value`	Inject single variable
`docker run --env-file .env`	Inject multiple from file

19.8.3 Monitoring

Command	Description
`docker stats`	Live resource usage
`docker top <container>`	process list inside container
`docker logs -f <container>`	Follow container logs

19.8.4 Networks

Command	Description
`docker network create <name>`	Create custom network
`docker run --network <name>`	Run container on network

19.9 Key Takeaways

1. **Volumes persist data** safely outside the container lifecycle.
2. **Env variables configure apps** without modifying the actual code or image.
3. **Monitoring helps debugging** performance, memory issues, and hidden errors.
4. **Networks enable communication** securely via DNS isolation between containers.
5. Use **named volumes in production** for database persistence.
6. Use **custom networks** for multi-container applications instead of linking containers manually.

19.10 □ Navigation

Previous	Home	Next
← Lab 4	Main README	Lab 6 →

19.10.1 All Labs

- Lab 1 — VM vs Container Comparison
- Lab 2 — Docker Installation & GitHub Pages
- Lab 3 — Docker Images, NGINX & Flask Deployment

- Lab 4 — Docker Essentials
- **Lab 5 — Volumes, Env Vars, Monitoring, Networks** *(You are here)*
- Lab 6 — Docker Run vs Docker Compose

20 Experiment 6A: Comparison of Docker Run and Docker Compose

20.1 PART A – THEORY

20.1.1 1. Objective

To understand the relationship between Docker Run and Docker Compose, and compare their syntax and use cases.

20.1.2 2. Background Theory

20.1.2.1 2.1 Docker Run (Imperative Approach)

- Used to create and start containers.
- Requires manual flags for everything (Ports, Volumes, Networks, Restart policies, Resources, Names).

Example:

```
docker run -d \
 --name my-nginx \
 -p 8080:80 \
 -v ./html:/usr/share/nginx/html \
 -e NGINX_HOST=localhost \
 --restart unless-stopped \
 nginx:alpine
```

20.1.2.2 2.2 Docker Compose (Declarative Approach)

- Uses `docker-compose.yml` to define services, networks, and volumes in a single file.
- One command (`docker compose up -d`) runs everything.

Equivalent Compose File:

```yaml
version: '3.8'
services:
  nginx:
    image: nginx:alpine
    container_name: my-nginx
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    environment:
```

```
      NGINX_HOST: localhost
    restart: unless-stopped
```

20.1.3 3. Mapping: Docker Run vs Compose

Docker Run	Docker Compose
`-p`	`ports`
`-v`	`volumes`
`-e`	`environment`
`--name`	`container_name`
`--network`	`networks`
`--restart`	`restart`
`--memory`	`deploy.resources.limits.memory`
`--cpus`	`deploy.resources.limits.cpus`

20.1.4 4. Advantages of Docker Compose

- Multi-container support
- Reproducibility & Version control
- Unified lifecycle commands
- Scaling capability (`docker compose up --scale web=3`)

20.2 PART B – PRACTICAL TASKS

20.2.1 Task 1: Single Container Comparison

20.2.1.1 Using Docker Run

```
docker run -d --name lab-nginx -p 8081:80 -v $(pwd)/html:/usr/share/nginx/html nginx:alpine
curl http://localhost:8081
```

20.2.1.2 Using Docker Compose docker-compose.yml

```
version: '3.8'
services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

```
docker compose up -d
docker compose ps
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps> cd c:\Users\nakul\OneDrive\Documents\Containerization-Dev
Ops\Lab\Lab4\flask-app
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app .
[+] Building 5.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 321B                                                                                   0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => [3/5] COPY requirements.txt .                                                                                   0.0s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                        4.8s
 => [5/5] COPY app.py .                                                                                             0.1s
 => exporting to image                                                                                              0.1s
 => => exporting layers                                                                                             0.1s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                        0.0s
 => => naming to docker.io/library/my-flask-app                                                                     0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ryzb5nsaan18zi1j8r1jfto3b
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 40: Docker Run Output

20.2.2 Task 2: Multi-Container App (WordPress + MySQL)

20.2.2.1 Using Docker Run (Manual approach)

```
docker network create wp-net

docker run -d --name mysql --network wp-net -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=wo

docker run -d --name wordpress --network wp-net -p 8082:80 -e WORDPRESS_DB_HOST=mysql -e WORDP
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 63B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                      0.0s
 => exporting to image                                                                                              0.0s
 => => exporting layers                                                                                             0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                        0.0s
 => => naming to docker.io/library/my-flask-app:1.0                                                                 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cvnruc9vjk8gembw1bx5isnan
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:latest -t my-flask-
app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 63B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                      0.0s
 => exporting to image                                                                                              0.0s
 => => exporting layers                                                                                             0.0s
 => => writing image                                                                                                0.0s
 => => naming to doc                                                                                                0.0s
 => => naming to doc                                                                                                0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/x6thk2vxzkbqba8dqivqetw0v
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker tag my-flask-app:latest my-flask-app:v1.0
```

docker-desktop://dashboard/build/desktop-linux/desktop-linux/x6thk2vxzkbqba8dqivqetw0v (ctrl + click)

Figure 41: Docker Compose Output

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab6\task1-single> docker network create wp-net
>>
4e7f92733d0d0919dfdf509abca1772c4dfd32e15817f5b0ed5d32e78ec5206e
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab6\task1-single> docker run -d --name mysql --network wp-net -e MYSQL_ROOT_PASSWORD
=secret -e MYSQL_DATABASE=wordpress mysql:5.7
>>
Unable to find image 'mysql:5.7' locally
5.7: Pulling from library/mysql
20e4dcae4c69: Pull complete
1c56c3d4ce74: Pull complete
e9f03a1c24ce: Pull complete
68c3898c2015: Pull complete
6b95a940e7b6: Pull complete
90986bb8de6e: Pull complete
5.7: Pulling from library/mysql
20e4dcae4c69: Pull complete
1c56c3d4ce74: Pull complete
e9f03a1c24ce: Pull complete
68c3898c2015: Pull complete
6b95a940e7b6: Pull complete
90986bb8de6e: Pull complete
20e4dcae4c69: Pull complete
1c56c3d4ce74: Pull complete
e9f03a1c24ce: Pull complete
68c3898c2015: Pull complete
6b95a940e7b6: Pull complete
90986bb8de6e: Pull complete
68c3898c2015: Pull complete
6b95a940e7b6: Pull complete
90986bb8de6e: Pull complete
6b95a940e7b6: Pull complete
90986bb8de6e: Pull complete
90986bb8de6e: Pull complete
ae71319cb779: Pull complete
ffc89e9dfd88: Pull complete
43d05e938198: Pull complete
064b2d298fba: Pull complete
43d05e938198: Pull complete
064b2d298fba: Pull complete
df9a4d85569b: Pull complete
Digest: sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb
064b2d298fba: Pull complete
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab6\task1-single> docker run -d --name wordpress --network wp-net -p 8082:80 -e WORD
PRESS_DB_HOST=mysql -e WORDPRESS_DB_PASSWORD=secret wordpress:latest
>>
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
ec781dee3f47: Already exists
492c65e7c8ac: Pull complete
60db4f438f38: Pull complete
7d5440bdc2a2: Pull complete
f0ff10d8b131: Pull complete
fa79359a5906: Pull complete
0e62964b059f: Pull complete
9ab4f7b41339: Pull complete
a6418fb4b014: Pull complete
b879b42dc415: Pull complete
7ddb4115d2ee: Pull complete
41d15dbe17b9: Pull complete
48a25387913f: Pull complete
5a5870c8f47c: Pull complete
4f4fb700ef54: Pull complete
3e00916808b5: Pull complete
ce09f597207b: Pull complete
021cf943304b: Pull complete
b890e1219e2a: Pull complete
afc573caa3a9: Pull complete
ce8606c13c15: Pull complete
5451487dab40: Pull complete
c61acaf42e5d: Pull complete
4413d269cb54: Pull complete
Digest: sha256:ee74dc0ebbe7ec43b7f473c0fb4f1dfb6b211d7785f0a95bcdfa10e628c2e67f
Status: Downloaded newer image for wordpress:latest
329a695e009b1572684f4802cdc0398666e6553d12e21fdeb736ae7641bc6538
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab6\task1-single> docker ps
>>
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS                                       NAMES
329a695e009b   wordpress:latest   "docker-entrypoint.s…"   12 seconds ago   Up 11 seconds   0.0.0.0:8082->80/tcp, [::]:8082->80/tcp     wordpress
5b51597fd921   mysql:5.7          "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes    3306/tcp, 33060/tcp                         mysql
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab6\task1-single>
```

(Note: The equivalent steps in Compose are demonstrated in Experiment 6B below).

20.3 PART C – CONVERSION TASKS (Theory)

Converting manual `docker run` commands and configurations into declarative `docker-compose.yml` configurations:

20.3.1 Task 3: Convert Run → Compose

Given Run Command:

```
docker run -d --name webapp -p 5000:5000 -e APP_ENV=production -e DEBUG=false --restart unless-
```

Converted Compose File:

```yaml
version: '3.8'
services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: false
    restart: unless-stopped
```

20.3.2 Task 4 & 5: Volumes, Networks, and Resource Limits

Compose clearly organizes dependencies, limits, and custom named networks natively without extra CLI scaffolding. *(See lab theory notes for full limit syntax using `deploy:`)*.

20.4 PART D – DOCKERFILE WITH COMPOSE

We can combine custom image building with Compose orchestration.

app.js

```javascript
const http = require('http');
http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
```

Dockerfile

```dockerfile
FROM node:18-alpine
WORKDIR /app
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```

docker-compose.yml

```yaml
version: '3.8'
services:
  nodeapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
```

```
docker compose up --build -d
curl http://localhost:3000
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab6\part-d-node> docker compose up --build -d
>>
#7 extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 0.1s done
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 4.19MB / 40.01MB 4.5s
#7 sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B 4.4s done
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 6.29MB / 40.01MB 5.3s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 8.39MB / 40.01MB 6.0s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 10.49MB / 40.01MB 7.1s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 12.58MB / 40.01MB 7.8s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 14.68MB / 40.01MB 8.4s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 16.78MB / 40.01MB 8.8s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 18.87MB / 40.01MB 9.3s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 20.97MB / 40.01MB 9.7s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 23.07MB / 40.01MB 10.2s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 25.17MB / 40.01MB 10.6s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 27.26MB / 40.01MB 11.0s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 29.36MB / 40.01MB 11.4s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 31.46MB / 40.01MB 11.9s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 33.55MB / 40.01MB 12.6s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 35.65MB / 40.01MB 13.2s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 37.75MB / 40.01MB 13.9s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB 14.1s
#7 sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB 14.1s done
#7 extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e
#7 extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 0.9s done
#7 extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3
#7 extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 0.0s done
#7 extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 done
#7 DONE 15.2s

#8 [2/3] WORKDIR /app
#8 DONE 0.3s

#9 [3/3] COPY app.js .
#9 DONE 0.0s

#10 exporting to image
#10 exporting layers 0.0s done
#10 writing image sha256:b2011ec73f826cf5ac5dbdeeed917339796fdf73fc9fccc49c491af2e7436e3f done
#10 naming to docker.io/library/part-d-node-nodeapp done
#10 DONE 0.0s

#11 resolving provenance for metadata file
#11 DONE 0.0s
[+] up 3/3
 ✓ Image part-d-node-nodeapp   Built                                                    21.4s
 ✓ Network part-d-node_default Created                                                   0.0s
 ✓ Container custom-node-app   Created                                                   0.0s
```

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab6\part-d-node> curl http://localhost:3000
>>

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): a


StatusCode        : 200
StatusDescription : OK
Content           : {68, 111, 99, 107...}
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Keep-Alive: timeout=5
                    Content-Length: 24
                    Date: Fri, 20 Mar 2026 05:43:56 GMT

                    Docker Compose Build Lab
Headers           : {[Connection, keep-alive], [Keep-Alive, timeout=5], [Content-Length, 24], [Date, Fri, 20 Mar 2026 05:43:56 GMT]}
RawContentLength  : 24



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab6\part-d-node> docker compose ps
>>
time="2026-03-20T11:14:04+05:30" level=warning msg="C:\\Users\\nakul\\OneDrive\\Documents\\Containerization-DevOps\\Lab\\Lab6\\part-d-node\\docker-com
pose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME              IMAGE                 COMMAND                  SERVICE   CREATED          STATUS          PORTS
custom-node-app   part-d-node-nodeapp   "docker-entrypoint.s…"   nodeapp   50 seconds ago   Up 50 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tc
p
```

---

21 Experiment 6B: WordPress + MySQL using Compose

Using a single Compose file completely replaces the manual networking and container execution from Task 2.

docker-compose.yml

```yaml
version: '3.9'
services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
```

```yaml
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

21.0.1 1. Run the Stack

```
docker compose up -d
docker ps
```

21.0.2 2. Scaling

```
docker compose up --scale wordpress=3
```

> **Important Learning Outcome:** The scaling intentionally throws an error because standard Docker Compose is limited here. Since `container_name: wordpress_app` and `ports: "8080:80"` are hardcoded in the file, Compose cannot launch 3 replicas that all want the identical name and host port. This demonstrates the exact scenario where **Docker Swarm** (which handles dynamic naming and load balances identical ports) becomes necessary for production!

21.1 Docker Compose vs Swarm Comparison

Feature	Compose	Swarm
Scope	Single host	Multi-node
Scaling	Manual / Limited by port bindings	Built-in
Load balancing	No	Yes
Self-healing	No	Yes

```
inerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:21 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : OK
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 2
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 20 Mar 2026 04:32:31 GMT
                    Server: Werkzeug/3.1.6 Python/3.9.25

                    OK
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:31 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED         STATUS         PORTS                                       NAMES
6f0107357708   my-flask-app   "python app.py"   7 minutes ago   Up 7 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

Figure 42: Running WordPress Stack

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t flask-regular .
 => [internal] load .dockerignore                                                                                    0.0s
 => => transferring context: 273B                                                                                    0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                     0.0s
 => [internal] load build context                                                                                    0.0s
 => => transferring context: 63B                                                                                     0.0s
 => CACHED [2/5] WORKDIR /app                                                                                        0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                             0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                  0.0s
 => CACHED [5/5] COPY app.py .                                                                                       0.0s
 => exporting to image                                                                                               0.0s
 => => exporting layers                                                                                              0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                         0.0s
 => => naming to docker.io/library/flask-regular                                                                     0.0s

 View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hcwinfxyz9znl4zzathqwj838
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -f Dockerfile.multistage -t flask-multistage .
>>
[+] Building 8.2s (13/13) FINISHED                                                                   docker:desktop-linux
 => [internal] load build definition from Dockerfile.multistage                                                      0.0s
 => => transferring dockerfile: 486B                                                                                 0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                   0.0s
 => [internal] load .dockerignore                                                                                    0.0s
 => => transferring context: 273B                                                                                    0.0s
 => [builder 1/5] FROM docker.io/library/python:3.9-slim                                                             0.0s
 => [internal] load build context                                                                                    0.0s
 => => transferring context: 63B                                                                                     0.0s
 => CACHED [builder 2/5] WORKDIR /app                                                                                0.0s
 => CACHED [builder 3/5] COPY requirements.txt .                                                                     0.0s
 => [builder 4/5] RUN python -m venv /opt/venv                                                                       3.4s
 => [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt                                                 3.3s
 => [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv                                                            0.2s
 => [stage-1 4/5] COPY app.py .                                                                                      0.0s
 => [stage-1 5/5] RUN useradd -m -u 1000 appuser                                                                     0.2s
 => exporting to image                                                                                               0.2s
 => => exporting layers                                                                                              0.2s
 => => writing image sha256:15e019e88dc649d3371adfd7203641fbae971a4cb776c52bb56e64a89bde403a                         0.0s
 => => naming to docker.io/library/flask-multistage                                                                  0.0s

 View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hm4w1vg4rgjqp4u2ou41tekcr
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images | findstr flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      15e019e88dc6        145MB             0B
flask-regular:latest         e42d57394cf0        132MB             0B
my-flask-app:1.0             e42d57394cf0        132MB             0B
my-flask-app:latest          e42d57394cf0        132MB             0B
my-flask-app:v1.0            e42d57394cf0        132MB             0B
nakul71/my-flask-app:1.0     0e08867af4d9        128MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 43: Scaling Error Output

21.2 Key Takeaways

1. **Compose simplifies multi-container apps** by keeping settings in version control.
2. **Declarative approach** is much easier than manually tracking `docker run` flags.
3. **Compose is excellent for dev/test**, but limited in native scaling due to port/name collisions without strict dynamic tuning.
4. **Docker Swarm** is much better for production cluster orchestration, load balancing, and self-healing.

21.3 Navigation

Previous	Home	Next
← Lab 5	Main README	Lab 7 →

21.3.1 All Labs

- Lab 1 — VM vs Container Comparison
- Lab 2 — Docker Installation & GitHub Pages
- Lab 3 — Docker Images, NGINX & Flask Deployment
- Lab 4 — Docker Essentials
- Lab 5 — Volumes, Env Vars, Monitoring, Networks
- **Lab 6 — Docker Run vs Docker Compose** *(You are here)*
- Lab 7 — CI/CD using Jenkins, GitHub & Docker Hub

22 Lab 7: CI/CD using Jenkins, GitHub & Docker Hub

22.1 1. Aim

To build a complete CI/CD pipeline using:

- **GitHub** — Source code repository
- **Jenkins** — Automation server
- **Docker Hub** — Container image storage

22.2 2. Objective

- Understand the CI/CD workflow end-to-end
- Automate the build and push process using Jenkins
- Write a Jenkins pipeline using `Jenkinsfile`
- Store credentials securely inside Jenkins
- Trigger automated builds using GitHub Webhooks

22.3 3. What is CI/CD?

22.3.1 CI – Continuous Integration

Code is automatically built and tested after every commit pushed to GitHub.

22.3.2 CD – Continuous Deployment

The tested application is automatically packaged and deployed without manual steps.

22.3.3 Flow Diagram

```
Developer → GitHub → Webhook → Jenkins → Docker Hub
```

22.4 4. What is Jenkins?

Jenkins is a **GUI-based open-source automation server** that enables:

- **Building** applications
- **Testing** code automatically
- **Deploying** to target environments

22.4.1 Key Features

Feature	Description
Web Dashboard	Visual interface to manage jobs and pipelines
Plugin Support	1800+ plugins for every major tool
Pipeline as Code	Define your full CI/CD flow in a `Jenkinsfile`
Credential Store	Securely handles secrets and tokens

22.5 5. Project Structure

```
my-app/
    app.py
    requirements.txt
    Dockerfile
    Jenkinsfile
```

22.6 6. Application Code (Flask)

app.py

```python
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"

app.run(host="0.0.0.0", port=80)
```

requirements.txt

```
flask
```

22.7 7. Dockerfile

```dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 80
CMD ["python", "app.py"]
```

22.8 8. Jenkinsfile (Pipeline)

The `Jenkinsfile` defines the full CI/CD pipeline as code:

```jenkinsfile
pipeline {
    agent any

    environment {
        IMAGE_NAME = "your-dockerhub-username/myapp"
    }

    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/your-username/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }
```

```
        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOK
                    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --passwor
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
    }
}
```

22.8.1 Pipeline Stages Explained

Stage	What It Does
`Clone Source`	Pulls the latest code from GitHub
`Build Docker Image`	Runs `docker build` to create the image
`Login to Docker Hub`	Authenticates using stored token credential
`Push to Docker Hub`	Uploads the built image to Docker Hub

22.9 9. Jenkins Setup using Docker

We run Jenkins itself as a Docker container for easy setup.

docker-compose.yml

```yaml
version: "3.8"
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

Why mount `/var/run/docker.sock`? This allows Jenkins (running inside a container) to communicate with the host Docker daemon — so it can build and push images directly from within the pipeline.

22.9.1 Starting Jenkins

```
docker compose up -d
```

Then open: **http://localhost:8080**

22.10 10. Jenkins Configuration

22.10.1 Step 1 – Add Docker Hub Credentials

Navigate to:

`Manage Jenkins → Credentials → System → Global credentials → Add Credentials`

- **Kind:** Secret text
- **ID:** `dockerhub-token`
- **Secret:** Your Docker Hub access token

22.10.2 Step 2 – Create a Pipeline Job

1. Click **New Item**
2. Select **Pipeline**
3. Under **Pipeline Definition**, select `Pipeline script from SCM`
4. Set **SCM** to Git and paste your GitHub repo URL
5. Jenkins will automatically pick up the `Jenkinsfile` from the repo root

---

22.11 11. Webhook Integration

22.11.1 In GitHub

1. Go to your repo → **Settings → Webhooks → Add webhook**
2. Set the Payload URL to:

   ```
   http://<your-jenkins-ip>:8080/github-webhook/
   ```

3. Content type: `application/json`
4. Trigger on: **Push events**

Now every `git push` will automatically trigger the Jenkins pipeline!

22.12 12. Execution Flow

```
Step 1: Developer pushes code to GitHub
         ↓
Step 2: GitHub sends a Webhook notification to Jenkins
         ↓
Step 3: Jenkins Pipeline runs:
        → Clone code
        → Build Docker image
        → Login to Docker Hub
        → Push image
         ↓
Step 4: Image is now live on Docker Hub
```

22.13 13. Key Concept: `withCredentials`

22.13.1 Problem

Hardcoding passwords in `Jenkinsfile` exposes secrets in version control.

22.13.2 Solution

```
withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdin'
}
```

Jenkins:

- Fetches the secret from the encrypted credentials store
- Injects it as an environment variable **temporarily**
- Automatically masks it from all logs
- Removes it from memory after the block completes

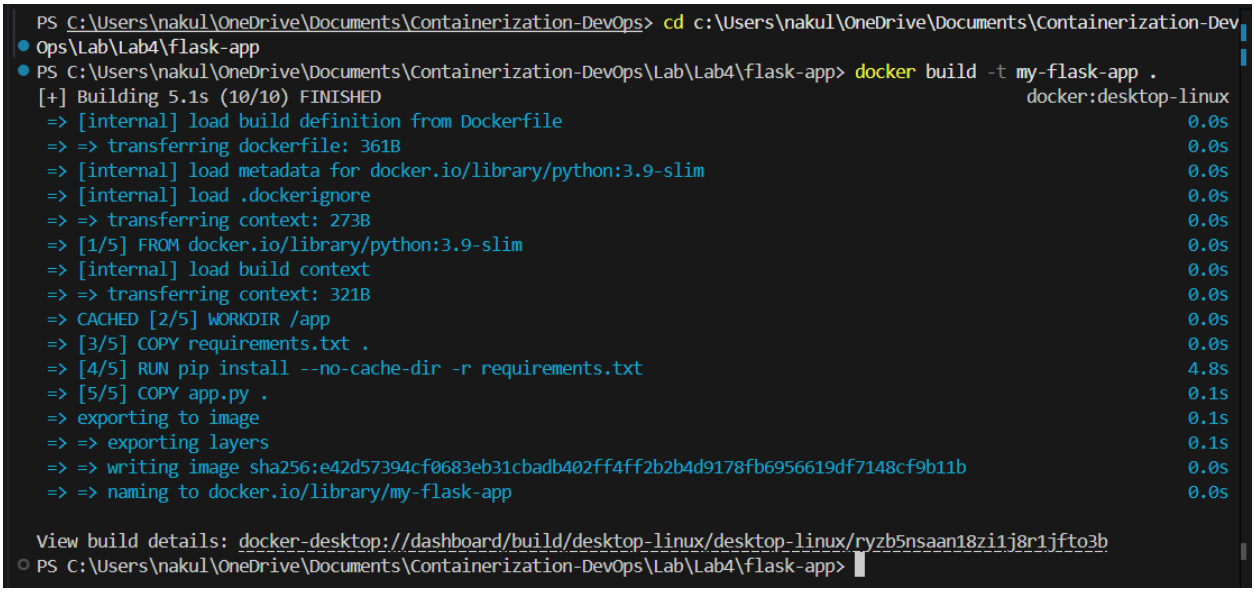
22.14 14. Key Commands in Pipeline

Command	Purpose
`git`	Clone the source code repository
`sh`	Execute a shell command on the agent
`docker build`	Build a Docker image from the `Dockerfile`
`docker login`	Authenticate with Docker Hub
`docker push`	Upload the image to Docker Hub

22.15 15. Observations & Screenshots

The following screenshots document the complete setup and execution of this CI/CD pipeline.

22.15.1 Jenkins Running via Docker Compose



```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps> cd c:\Users\nakul\OneDrive\Documents\Containerization-Dev
Ops\Lab\Lab4\flask-app
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app .
[+] Building 5.1s (10/10) FINISHED                                                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                       0.0s
 => => transferring dockerfile: 361B                                                                       0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                         0.0s
 => [internal] load .dockerignore                                                                          0.0s
 => => transferring context: 273B                                                                          0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                           0.0s
 => [internal] load build context                                                                          0.0s
 => => transferring context: 321B                                                                          0.0s
 => CACHED [2/5] WORKDIR /app                                                                              0.0s
 => [3/5] COPY requirements.txt .                                                                          0.0s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                               4.8s
 => [5/5] COPY app.py .                                                                                    0.1s
 => exporting to image                                                                                     0.1s
 => => exporting layers                                                                                    0.1s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b               0.0s
 => => naming to docker.io/library/my-flask-app                                                            0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ryzb5nsaan18zi1j8r1jfto3b
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 44: Jenkins Docker Compose Up

22.15.2 Jenkins Initial Setup & Dashboard

22.15.3 Adding Docker Hub Credentials in Jenkins

22.15.4 Pipeline Job Configuration

22.15.5 Pipeline Build Execution

22.15.6 Image Successfully Pushed to Docker Hub

22.16 16. Result

Successfully created a complete automated CI/CD pipeline:

Component	Role
GitHub	Stores source code, triggers webhook on push
Jenkins	Automates build, login, and push stages
Docker	Provides consistent build environment
Docker Hub	Stores and distributes the built image

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 63B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                      0.0s
 => exporting to image                                                                                              0.0s
 => => exporting layers                                                                                             0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                        0.0s
 => => naming to docker.io/library/my-flask-app:1.0                                                                 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cvnruc9vjk8gembw1bx5isnan
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:latest -t my-flask-
app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 361B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  0.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 273B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                    0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 63B                                                                                    0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY app.py .                                                                                      0.0s
 => exporting to image                                                                                              0.0s
 => => exporting layers                                                                                             0.0s
 => => writing image                                                                                                0.0s
 => => naming to doc                                                                                                0.0s
 => => naming to doc                                                                                                0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/x6thk2vxzkbqba8dqivqetw0v
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker tag my-flask-app:latest my-flask-app:v1.0
```

docker-desktop://dashboard/build/desktop-linux/desktop-linux/x6thk2vxzkbqba8dqivqetw0v (ctrl + click)

Figure 45: Jenkins Dashboard

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
assignment1-backend:latest     696e1abfeb2b        185MB             0B
assignment1-db:latest          7068f884d33f        276MB             0B
experiment4:1.0                0e08867af4d9        128MB             0B
experiment4:latest             4231c3a2811e        132MB             0B    U
experiment:1.0                 0e08867af4d9        128MB             0B
flask-multistage:latest        94a39ca50667        145MB             0B
flask-regular:latest           4231c3a2811e        132MB             0B    U
my-flask-app:1.0               e42d57394cf0        132MB             0B
my-flask-app:latest            e42d57394cf0        132MB             0B
my-flask-app:v1.0              e42d57394cf0        132MB             0B
mysql:8                        a0f6c7786c75        777MB             0B    U
mysql:8.0                      6f19538dd7d2        780MB             0B
nakul71/my-flask-app:1.0       0e08867af4d9        128MB             0B
nginx-alpine:latest            a33d5a30a4a0       10.4MB             0B
nginx-ubuntu:latest            5363ea9a780d        207MB             0B
nginx:latest                   5cdef4ac3335        161MB             0B
python:3.9-slim                085da638e1b8        122MB             0B
ubuntu:latest                  493218ed0f40       78.1MB             0B
wordpress:latest               6fe8e9457b10        752MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker history my-flask-app
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e42d57394cf0   11 minutes ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY app.py . # buildkit                        228B      buildkit.dockerfile.v0
<missing>      11 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   10.5MB    buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY requirements.txt . # buildkit              13B       buildkit.dockerfile.v0
<missing>      3 weeks ago      WORKDIR /app                                    0B        buildkit.dockerfile.v0
<missing>      4 months ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  for src in idle3 p…   36B       buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…   39.5MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  apt-get update;  a…   3.81MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   78.6MB    debuerreotype 0.16
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
>> docker inspect my-flask-app
[
    {
        "Id": "sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b",
        "RepoTags": [
            "my-flask-app:1.0",
            "my-flask-app:latest",
            "my-flask-app:v1.0"
        ],
```

Figure 46: Jenkins Credentials

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker run -d -p 5000:5000 --name flask-containe
r my-flask-app
>>
6f01073577084341c00e87d2de6ff354c17dc102264f25649799bc31c2a11c49
```

Figure 47: Pipeline Job Setup

```
inerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:21 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : OK
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 2
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 20 Mar 2026 04:32:31 GMT
                    Server: Werkzeug/3.1.6 Python/3.9.25

                    OK
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:31 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED         STATUS         PORTS                                       NAMES
6f0107357708   my-flask-app   "python app.py"   7 minutes ago   Up 7 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

Figure 48: Pipeline Running

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t flask-regular .
 => [internal] load .dockerignore                                                                                    0.0s
 => => transferring context: 273B                                                                                    0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim                                                                     0.0s
 => [internal] load build context                                                                                    0.0s
 => => transferring context: 63B                                                                                     0.0s
 => CACHED [2/5] WORKDIR /app                                                                                        0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                             0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                  0.0s
 => CACHED [5/5] COPY app.py .                                                                                       0.0s
 => exporting to image                                                                                               0.0s
 => => exporting layers                                                                                              0.0s
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b                         0.0s
 => => naming to docker.io/library/flask-regular                                                                     0.0s

 View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hcwinfxyz9znl4zzathqwj838
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -f Dockerfile.multistage -t flask-multistage .
>>
[+] Building 8.2s (13/13) FINISHED                                                                   docker:desktop-linux
 => [internal] load build definition from Dockerfile.multistage                                                      0.0s
 => => transferring dockerfile: 486B                                                                                 0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                   0.0s
 => [internal] load .dockerignore                                                                                    0.0s
 => => transferring context: 273B                                                                                    0.0s
 => [builder 1/5] FROM docker.io/library/python:3.9-slim                                                             0.0s
 => [internal] load build context                                                                                    0.0s
 => => transferring context: 63B                                                                                     0.0s
 => CACHED [builder 2/5] WORKDIR /app                                                                                0.0s
 => CACHED [builder 3/5] COPY requirements.txt .                                                                     0.0s
 => [builder 4/5] RUN python -m venv /opt/venv                                                                       3.4s
 => [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt                                                 3.3s
 => [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv                                                            0.2s
 => [stage-1 4/5] COPY app.py .                                                                                      0.0s
 => [stage-1 5/5] RUN useradd -m -u 1000 appuser                                                                     0.2s
 => exporting to image                                                                                               0.2s
 => => exporting layers                                                                                              0.2s
 => => writing image sha256:15e019e88dc649d3371adfd7203641fbae971a4cb776c52bb56e64a89bde403a                         0.0s
 => => naming to docker.io/library/flask-multistage                                                                  0.0s

 View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hm4w1vg4rgjqp4u2ou41tekcr
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images | findstr flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      15e019e88dc6        145MB             0B
flask-regular:latest         e42d57394cf0        132MB             0B
my-flask-app:1.0             e42d57394cf0        132MB             0B
my-flask-app:latest          e42d57394cf0        132MB             0B
my-flask-app:v1.0            e42d57394cf0        132MB             0B
nakul71/my-flask-app:1.0     0e08867af4d9        128MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

Figure 49: Docker Hub Image

Every `git push` now **automatically** results in a new Docker image on Docker Hub — with **zero manual steps**.

22.17 Key Takeaways

1. **Jenkins automates** the entire pipeline — no manual intervention needed after setup.
2. **GitHub stores code** and acts as the trigger source via webhooks.
3. **Docker ensures consistency** — the same image runs anywhere.
4. `withCredentials` safely handles secrets without exposing them in logs or code.
5. **Pipeline as Code** (`Jenkinsfile`) keeps your CI/CD logic version-controlled alongside your app.

22.18 Navigation

Previous	Home	Next
← Lab 6	Main README	—

22.18.1 All Labs

- Lab 1 — VM vs Container Comparison
- Lab 2 — Docker Installation & GitHub Pages
- Lab 3 — Docker Images, NGINX & Flask Deployment
- Lab 4 — Docker Essentials
- Lab 5 — Volumes, Env Vars, Monitoring, Networks
- Lab 6 — Docker Run vs Docker Compose
- **Lab 7 — CI/CD using Jenkins, GitHub & Docker Hub** *(You are here)*

23 Experiment 9: Ansible – Execution Prompt

23.1 Objective

To automate configuration and management of multiple servers using Ansible playbooks.

23.2 Prerequisites

- Docker Installed and running.
- WSL (Ubuntu) to safely run Ansible as a Control Node.
- Python 3 installed.

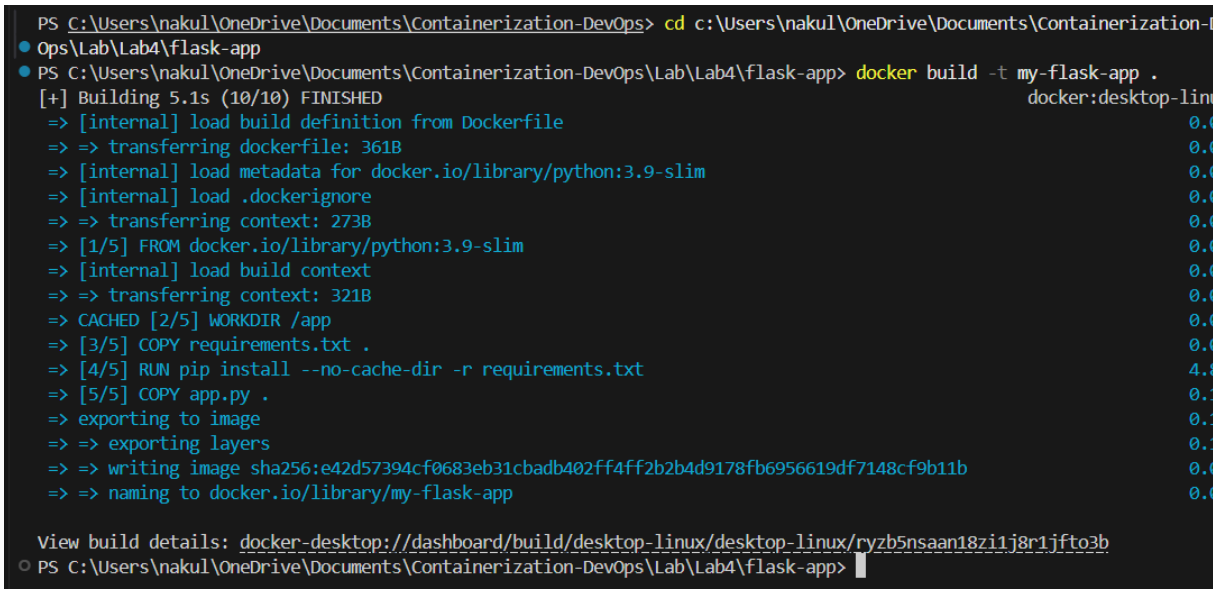
23.3 Steps Performed

23.3.1 1. Install Ansible

Ansible was installed inside the Ubuntu WSL environment:

```
sudo apt update -y
sudo apt install ansible -y
ansible --version
```

Screenshot:



23.3.2 2. Test Ansible Installation

Tested base functionality:

```
ansible localhost -m ping
```

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                    docker:desktop-lin
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 361B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load .dockerignore
 => => transferring context: 273B
 => [1/5] FROM docker.io/library/python:3.9-slim
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b
 => => naming to docker.io/library/my-flask-app:1.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cvnruc9vjk8gembw1bx5isnan
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:latest -t my-fla
app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                    docker:desktop-lin
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 361B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load .dockerignore
 => => transferring context: 273B
 => [1/5] FROM docker.io/library/python:3.9-slim
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => writing image
 => => naming to doc
 => => naming to doc
                    docker-desktop://dashboard/build/desktop-linux/desktop-
                    linux/x6thk2vxzkbqba8dqivqetw0v (ctrl + click)

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/x6thk2vxzkbqba8dqivqetw0v
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker tag my-flask-app:latest my-flask-app:
```

23.3.3 3. Generate SSH Key Pair

Ansible relies on SSH. An RSA Key Pair was created to securely connect to target servers without password prompts:

```
ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```

23.3.4 4. Create Dockerfile (Ubuntu SSH Server)

To simulate servers to manage, a `Dockerfile` was created defining an Ubuntu image with SSH and Python installed:

```
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd
```

```
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config &&
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh

COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

RUN chmod 600 /root/.ssh/id_rsa && chmod 644 /root/.ssh/authorized_keys

RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pa

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]
```

23.3.5 5. Build Docker Image

Built our mock server image:

```
docker build -t ubuntu-server .
```

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images
IMAGE                          ID            DISK USAGE   CONTENT SIZE   EXTRA
assignment1-backend:latest     696e1abfeb2b       185MB             0B
assignment1-db:latest          7068f884d33f       276MB             0B
experiment4:1.0                0e08867af4d9       128MB             0B
experiment4:latest             4231c3a2811e       132MB             0B    U
experiment:1.0                 0e08867af4d9       128MB             0B
flask-multistage:latest        94a39ca50667       145MB             0B
flask-regular:latest           4231c3a2811e       132MB             0B    U
my-flask-app:1.0               e42d57394cf0       132MB             0B
my-flask-app:latest            e42d57394cf0       132MB             0B
my-flask-app:v1.0              e42d57394cf0       132MB             0B
mysql:8                        a0f6c7786c75       777MB             0B    U
mysql:8.0                      6f19538dd7d2       780MB             0B
nakul71/my-flask-app:1.0       0e08867af4d9       128MB             0B
nginx-alpine:latest            a33d5a30a4a0      10.4MB             0B
nginx-ubuntu:latest            5363ea9a780d       207MB             0B
nginx:latest                   5cdef4ac3335       161MB             0B
python:3.9-slim                085da638e1b8       122MB             0B
ubuntu:latest                  493218ed0f40      78.1MB             0B
wordpress:latest               6fe8e9457b10       752MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker history my-flask-app
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e42d57394cf0   11 minutes ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY app.py . # buildkit                        228B      buildkit.dockerfile.v0
<missing>      11 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   10.5MB    buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY requirements.txt . # buildkit              13B       buildkit.dockerfile.v0
<missing>      3 weeks ago      WORKDIR /app                                    0B        buildkit.dockerfile.v0
<missing>      4 months ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  for src in idle3 p…   36B       buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…   39.5MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  apt-get update;  a…   3.81MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   78.6MB    debuerreotype 0.16
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
>> docker inspect my-flask-app
[
    {
        "Id": "sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b",
        "RepoTags": [
            "my-flask-app:1.0",
            "my-flask-app:latest",
            "my-flask-app:v1.0"
        ],
```

23.3.6 6. Run Multiple Servers

Deployed four mock servers using Docker:

```
for i in {1..4}; do
  docker run -d -p 220${i}:22 --name server${i} ubuntu-server
done
```

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker run -d -p 5000:5000 --name flask-cont
r my-flask-app
>>
6f01073577084341c00e87d2de6ff354c17dc102264f25649799bc31c2a11c49
```

23.3.7 7. Create Inventory File

Extracted the IPs of the Docker containers and generated an `inventory.ini` mapping so Ansible knows exactly how to connect to them:

```
[servers]
<server_ip_1>
<server_ip_2>
<server_ip_3>
<server_ip_4>

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

23.3.8 8. Test Connectivity

Ensured Ansible could cleanly reach all target machines:

```
ansible all -i inventory.ini -m ping
```

Screenshot:

```
inerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:21 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18


PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : OK
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 2
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 20 Mar 2026 04:32:31 GMT
                    Server: Werkzeug/3.1.6 Python/3.9.25

                    OK
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:31 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2


PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED         STATUS         PORTS                                       NAMES
6f0107357708   my-flask-app   "python app.py"   7 minutes ago   Up 7 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

23.3.9 9. Create Playbook

Wrote an Ansible Playbook (`playbook.yml`) to automatically install packages and deploy a text file to all remote nodes.

```yaml
---
- name: Configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update packages
      apt:
        update_cache: yes

    - name: Install packages
      apt:
        name: ["vim", "htop", "wget"]
        state: present

    - name: Create file
      copy:
        dest: /root/test.txt
        content: "Configured by Ansible on {{ inventory_hostname }}\n"
```

23.3.10 10. Run Playbook

Executed the playbook against all four dummy servers:

```
ansible-playbook -i inventory.ini playbook.yml
```

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t flask-regular .
 => [internal] load .dockerignore
 => => transferring context: 273B
 => [1/5] FROM docker.io/library/python:3.9-slim
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b
 => => naming to docker.io/library/flask-regular

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hcwinfxyz9znl4zzathqwj838
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -f Dockerfile.multistage -t flask-multistage .
>>
[+] Building 8.2s (13/13) FINISHED                                                                          docker:desktop-li
 => [internal] load build definition from Dockerfile.multistage
 => => transferring dockerfile: 486B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load .dockerignore
 => => transferring context: 273B
 => [builder 1/5] FROM docker.io/library/python:3.9-slim
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [builder 2/5] WORKDIR /app
 => CACHED [builder 3/5] COPY requirements.txt .
 => [builder 4/5] RUN python -m venv /opt/venv
 => [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt
 => [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv
 => [stage-1 4/5] COPY app.py .
 => [stage-1 5/5] RUN useradd -m -u 1000 appuser
 => exporting to image
 => => exporting layers
 => => writing image sha256:15e019e88dc649d3371adfd7203641fbae971a4cb776c52bb56e64a89bde403a
 => => naming to docker.io/library/flask-multistage

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hm4w1vg4rgjqp4u2ou41tekcr
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images | findstr flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      15e019e88dc6        145MB             0B
flask-regular:latest         e42d57394cf0        132MB             0B
my-flask-app:1.0             e42d57394cf0        132MB             0B
my-flask-app:latest          e42d57394cf0        132MB             0B
my-flask-app:v1.0            e42d57394cf0        132MB             0B
nakul71/my-flask-app:1.0     0e08867af4d9        128MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

23.3.11 11. Verify Output

Remotely checked the outcome on the target servers:

```
ansible all -i inventory.ini -m command -a "cat /root/test.txt"
```

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker build -t my-node-app .
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
 => => sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 7.67kB / 7.67kB
 => => sha256:929b04d7c782f04f615cf785488fed452b6569f87c73ff666ad553a7554f0006 1.72kB / 1.72kB
 => => sha256:ee77c6cd7c1886ecc802ad6cedef3a8ec1ea27d1fb96162bf03dd3710839b8da 6.18kB / 6.18kB
 => => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
 => => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB
 => => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB
 => => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870
 => => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B
 => => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e
 => => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3
 => => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549
 => [internal] load build context
 => => transferring context: 508B
 => [2/5] WORKDIR /app
 => [3/5] COPY package*.json ./
 => [4/5] RUN npm install --only=production
 => [5/5] COPY app.js .
 => exporting to image
 => => exporting layers
 => => writing image sha256:162988703b5ad65bd171df66945c0a97285c3b434f17c4d83885c9cec00b49d8
 => => naming to docker.io/library/my-node-app

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/4496y9gfu9ztx4o7wy9owiw8u
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> docker run -d -p 3000:3000 --name node-container my-node-app
670297bb798dfb673d23854fd019a82e0845ec16ed963ebdca89cb2762d9dfde
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\node-app> curl http://localhost:3000

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : Hello from Node.js Docker!
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Keep-Alive: timeout=5
                    Content-Length: 26
                    Content-Type: text/html; charset=utf-8
```

23.3.12 12. Cleanup

Removed all test simulation containers:

```
for i in {1..4}; do
  docker rm -f server${i}
done
```

23.4 Results

- Ansible successfully initialized and mapped to multi-node targets dynamically.
- `inventory.ini` effectively permitted connection overriding local Windows permissions mappings.
- The `playbook.yml` securely provisioned 4 nodes installing `vim`, `htop`, `wget` securely without failing.
- A dynamically verified file `/root/test.txt` proved environment replication and template integration!

24 Experiment 10: SonarQube – Static Code Analysis

24.1 1. Aim

To analyze code quality using SonarQube and integrate it into a CI/CD pipeline.

24.2 2. Problem Statement

- Bugs found late
- Manual code review is slow
- Hard to maintain quality

24.3 3. What is SonarQube?

SonarQube is a tool that: - Scans code - Finds bugs - Detects security issues - Improves maintainability

 It uses **static analysis** (no execution required)

24.4 4. What Problems It Solves

- Detects issues early
- Ensures code quality
- Tracks technical debt
- Enforces quality rules

24.5 5. Key Terms (VERY IMPORTANT)

Term	Meaning
Bug	Code that may fail
Vulnerability	Security risk
Code Smell	Bad coding practice
Quality Gate	Pass/fail check
Technical Debt	Effort to fix issues
Coverage	Tested code %
Duplication	Repeated code

24.6 6. Architecture (VERY IMPORTANT)

According to the diagram: **Code → Scanner → SonarQube Server → Database → Dashboard**

24.6.1 Components

1. **SonarQube Server (Brain)**: Runs on port 9000, stores results, shows dashboard.

2. **Sonar Scanner (Worker)**: Reads code, finds issues, sends report.
3. **PostgreSQL**: Stores analysis data.

24.7 7. Important Concept

 Both are required: | Setup | Result | |—|—| | Only Server | No analysis | | Only Scanner | No storage | | **Both** | **Works** |

24.8 8. Token Authentication (IMPORTANT)

Flow: Generate Token → Pass to Scanner → Server verifies → Stores results Token = secure login (not password)

24.9 9. Step-by-Step Flow

24.9.1 Step 1: Start SonarQube Server

We start SonarQube and PostgreSQL.

```
docker-compose up -d
docker logs -f sonarqube
```

Open: http://localhost:9000

24.9.2 Terminal Output

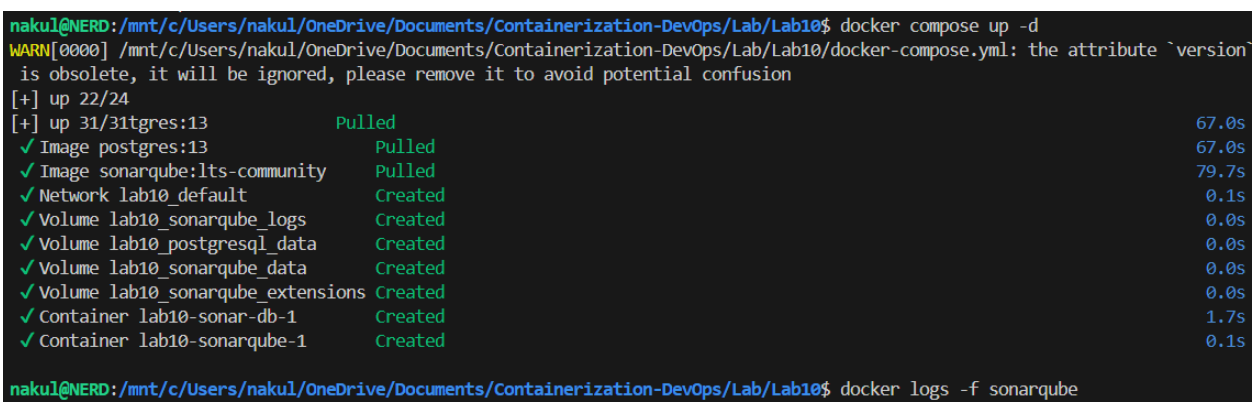


Figure 50: SonarQube Startup

24.9.3 Step 2: Sample Code & Run Scanner

First, ensure you have some code with bugs (Division by zero, SQL injection, unused variables). Then run the scanner against your code using the token generated from SonarQube.

Option 1: Maven

```
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

Option 2: CLI (Docker)

```
docker run -e SONAR_HOST_URL="http://localhost:9000" -e SONAR_LOGIN="YOUR_TOKEN" -v "${PWD}:/us
```

24.9.4 Terminal Output

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab10> wsl
2026.04.18 09:12:35 INFO  web[][o.s.s.p.w.MasterServletFilter] Initializing servlet filter org.sonar.server.plugins.PluginsRis
kConsentFilter@3cf83c30 [pattern=UrlPattern{inclusions=[/*], exclusions=[*.css, ...]}]
2026.04.18 09:12:36 INFO  web[][o.s.s.q.ProjectsInWarningDaemon] Counting number of projects in warning is not started as ther
e are no projects in this situation.
2026.04.18 09:12:36 INFO  web[][o.s.s.p.p.PlatformLevelStartup] Running Community Edition
2026.04.18 09:12:36 INFO  app[][o.s.a.SchedulerImpl] Process[web] is up
2026.04.18 09:12:36 INFO  app[][o.s.a.ProcessLauncherImpl] Launch process[COMPUTE_ENGINE] from [/opt/sonarqube]: /opt/java/ope
njdk/bin/java -Djava.awt.headless=true -Dfile.encoding=UTF-8 -Djava.io.tmpdir=/opt/sonarqube/temp -XX:-OmitStackTraceInFastThr
ow --add-opens=java.base/java.util=ALL-UNNAMED --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED --add-opens=java.base/java
.lang=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED --add-opens=java.man
agement/sun.management=ALL-UNNAMED --add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED -Dcom.redhat.fips=false
-Xmx512m -Xms128m -XX:+HeapDumpOnOutOfMemoryError -Dhttp.nonProxyHosts=localhost|127.*|[::1] -cp ./lib/sonar-application-9.9.8
.100196.jar:/opt/sonarqube/lib/jdbc/postgresql/postgresql-42.5.1.jar org.sonar.ce.app.CeServer /opt/sonarqube/temp/sq-process1
8062425993340646076properties
2026.04.18 09:12:36 WARN  app[][startup] ##################################################################################
################################
2026.04.18 09:12:36 WARN  app[][startup] Default Administrator credentials are still being used. Make sure to change the passw
ord or deactivate the account.
2026.04.18 09:12:36 WARN  app[][startup] ##################################################################################
################################
2026.04.18 09:12:36 INFO  web[][o.s.s.p.Platform] Web Server is operational
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by org.sonar.process.PluginSecurityManager (file:/opt/sonarqube/lib/sonar-
application-9.9.8.100196.jar)
WARNING: Please consider reporting this to the maintainers of org.sonar.process.PluginSecurityManager
WARNING: System::setSecurityManager will be removed in a future release
2026.04.18 09:12:36 INFO  ce[][o.s.p.ProcessEntryPoint] Starting Compute Engine
2026.04.18 09:12:36 INFO  ce[][o.s.ce.app.CeServer] Compute Engine starting up...
2026.04.18 09:12:37 INFO  ce[][o.sonar.db.Database] Create JDBC data source for jdbc:postgresql://sonar-db:5432/sonarqube
2026.04.18 09:12:37 INFO  ce[][c.z.h.HikariDataSource] HikariPool-1 - Starting...
2026.04.18 09:12:37 INFO  ce[][c.z.h.p.HikariPool] HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@25b28ede
2026.04.18 09:12:37 INFO  ce[][c.z.h.HikariDataSource] HikariPool-1 - Start completed.
2026.04.18 09:12:38 INFO  ce[][o.s.s.p.ServerFileSystemImpl] SonarQube home: /opt/sonarqube
2026.04.18 09:12:38 INFO  ce[][o.s.c.c.CePluginRepository] Load plugins
2026.04.18 09:12:39 INFO  ce[][o.s.c.c.ComputeEngineContainerImpl] Running Community edition
2026.04.18 09:12:39 INFO  ce[][o.s.ce.app.CeServer] Compute Engine is started
2026.04.18 09:12:39 INFO  app[][o.s.a.SchedulerImpl] Process[ce] is up
2026.04.18 09:12:39 INFO  app[][o.s.a.SchedulerImpl] SonarQube is operational
^Cnakul@NERD:/mnt/c/Users/nakul/OneDrive/Documents/Containerization-DevOps/Lab/Lab10$
```

Figure 51: Run Scanner

24.9.5 Step 3: View Dashboard Results

Open: http://localhost:9000 You'll see: Bugs, Vulnerabilities, Code Smells, Quality Gate.

Example Dashboard: - Bugs: 5 - Vulnerabilities: 1 - Code Smells: 8 - Quality Gate: FAILED

24.9.6 Terminal Output

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images
IMAGE                        ID             DISK USAGE   CONTENT SIZE   EXTRA
assignment1-backend:latest   696e1abfeb2b       185MB             0B
assignment1-db:latest        7068f884d33f       276MB             0B
experiment4:1.0              0e08867af4d9       128MB             0B
experiment4:latest           4231c3a2811e       132MB             0B    U
experiment:1.0               0e08867af4d9       128MB             0B
flask-multistage:latest      94a39ca50667       145MB             0B
flask-regular:latest         4231c3a2811e       132MB             0B    U
my-flask-app:1.0             e42d57394cf0       132MB             0B
my-flask-app:latest          e42d57394cf0       132MB             0B
my-flask-app:v1.0            e42d57394cf0       132MB             0B
mysql:8                      a0f6c7786c75       777MB             0B    U
mysql:8.0                    6f19538dd7d2       780MB             0B
nakul71/my-flask-app:1.0     0e08867af4d9       128MB             0B
nginx-alpine:latest          a33d5a30a4a0      10.4MB             0B
nginx-ubuntu:latest          5363ea9a780d       207MB             0B
nginx:latest                 5cdef4ac3335       161MB             0B
python:3.9-slim              085da638e1b8       122MB             0B
ubuntu:latest                493218ed0f40      78.1MB             0B
wordpress:latest             6fe8e9457b10       752MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker history my-flask-app
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e42d57394cf0   11 minutes ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY app.py . # buildkit                        228B      buildkit.dockerfile.v0
<missing>      11 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   10.5MB    buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY requirements.txt . # buildkit              13B       buildkit.dockerfile.v0
<missing>      3 weeks ago      WORKDIR /app                                    0B        buildkit.dockerfile.v0
<missing>      4 months ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  for src in idle3 p…   36B       buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…   39.5MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  apt-get update;  a…   3.81MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   78.6MB    debuerreotype 0.16
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
>> docker inspect my-flask-app
[
    {
        "Id": "sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b",
        "RepoTags": [
            "my-flask-app:1.0",
            "my-flask-app:latest",
            "my-flask-app:v1.0"
        ],
```

Figure 52: SonarQube Dashboard

24.9.7 Step 4: Automating via API

You can query issues programmatically using the SonarQube API.

```
curl -u admin:YOUR_TOKEN "http://localhost:9000/api/issues/search"
```

24.9.8 Terminal Output

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker run -d -p 5000:5000 --name flask-containe
r my-flask-app
>>
6f01073577084341c00e87d2de6ff354c17dc102264f25649799bc31c2a11c49
```

Figure 53: API Output

24.9.9 Step 5: Jenkins Pipeline Integration

Pipeline Flow: Checkout → Scan → Build → Deploy

We updated the `Jenkinsfile` from **Lab 7** to include a new stage for SonarQube. This ties both labs together!

Important Code Snippet added to Lab 7 Jenkinsfile:

```
stage('SonarQube Analysis') {
    steps {
        sh 'docker run --rm -v "${WORKSPACE}:/usr/src" sonarsource/sonar-scanner-cli -
    }
}
```

 Jenkins intercepts the code, executes the scanner locally, and visually displays whether the code allowed the pipeline to proceed to Docker Hub!

24.9.10 Terminal Output

24.10 15. Tool Comparison (Important Theory)

Tool	Purpose
Jenkins	CI/CD
SonarQube	Code quality
Ansible	Config management
Chef	Infra automation

24.11 16. Best Practices

- **Security**: Never hardcode tokens, use environment variables.
- **Code Quality**: Fix issues early, use Quality Gates.
- **CI/CD**: Run scan on every commit, fail pipeline on bad code.

24.12 17. Final Summary

- **SonarQube** = analysis platform
- **Scanner** = analyzes code
- **Token** = secure authentication
- **Quality Gate** = stops bad code

```
inerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:21 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : OK
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 2
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 20 Mar 2026 04:32:31 GMT
                    Server: Werkzeug/3.1.6 Python/3.9.25

                    OK
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:31 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED         STATUS         PORTS                                       NAMES
6f0107357708   my-flask-app   "python app.py"   7 minutes ago   Up 7 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

Figure 54: Jenkins Quality Gate Failure

24.13 FINAL UNDERSTANDING

SonarQube = "System that checks your code quality before it is deployed."

24.13.1 Important Insight for YOU

Now your learning stack is: **Docker → Compose → Kubernetes → Jenkins → SonarQube** *This is complete DevOps pipeline knowledge!*

25 Experiment 11: Orchestration using Docker Compose & Docker Swarm

25.1 1. Aim

To orchestrate multi-container applications using Docker Swarm, extending the WordPress + MySQL setup from Experiment 6 with scaling, self-healing, and load balancing.

25.2 2. Problem Statement

Docker Compose works well for development on a single machine, but it lacks: - Automatic recovery when containers crash - Built-in load balancing across replicas - Multi-host deployment capability

Docker Swarm solves these limitations by adding orchestration on top of Compose.

25.3 3. Concepts

25.3.1 The Progression Path

```
docker run  →  Docker Compose  →  Docker Swarm  →  Kubernetes

 Single         Multi-container    Orchestration    Advanced
 container      (single host)       (basic)        orchestration
```

25.3.2 What Orchestration Adds

Feature	What it means
Scaling	Increase/decrease number of containers with one command
Self-healing	Restart failed containers automatically
Load balancing	Distribute traffic across all replicas
Multi-host	Run containers across multiple machines

25.3.3 Compose vs Swarm

Feature	Docker Compose	Docker Swarm
Scope	Single host	Multi-node cluster
Scaling	Basic (`--scale`), port conflicts	`docker service scale`, no conflicts
Load Balancing	No	Yes (internal LB)
Self-Healing	No	Yes (automatic)
Rolling Updates	No	Yes (zero downtime)
Use Case	Development, testing	Simple production clusters

25.4 4. Prerequisites

- Docker installed with Swarm mode available
- `docker-compose.yml` from this directory (WordPress + MySQL)

25.5 5. Steps Performed

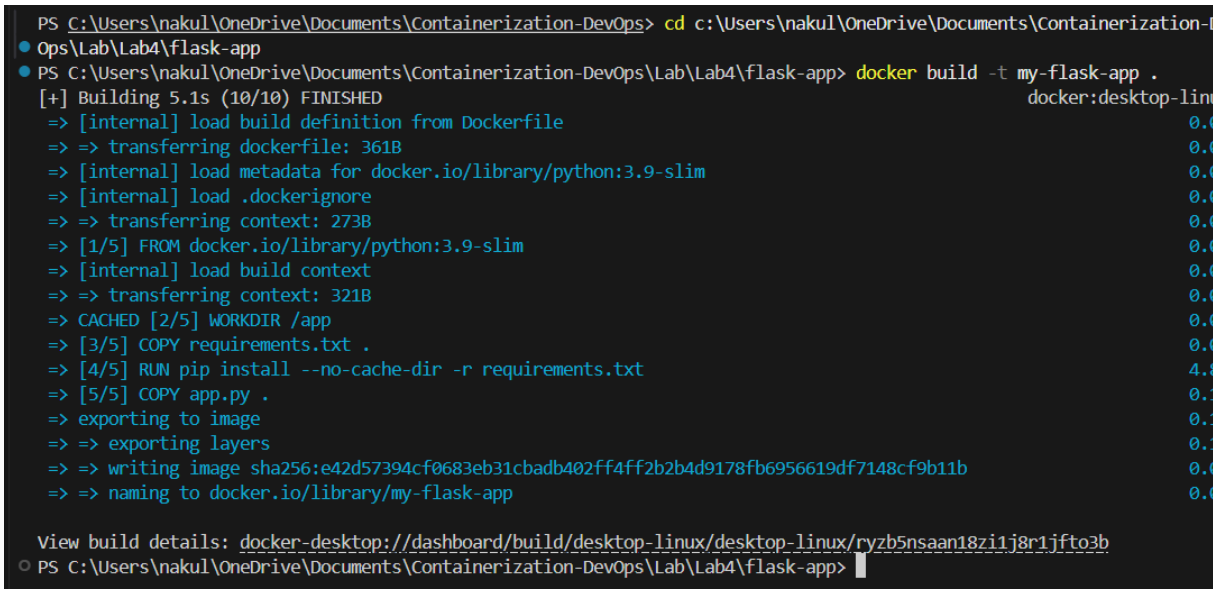
25.5.1 Task 1: Check Current State

Navigated to the Lab11 directory and stopped any existing Compose containers:

```
cd /mnt/c/Users/nakul/OneDrive/Documents/Containerization-DevOps/Lab/Lab11
docker compose down -v
```

Output confirmed Docker version `28.2.2, build 28.2.2-0ubuntu1~24.04.1` and no containers were running.

Screenshot:



25.5.2 Task 2: Initialize Docker Swarm

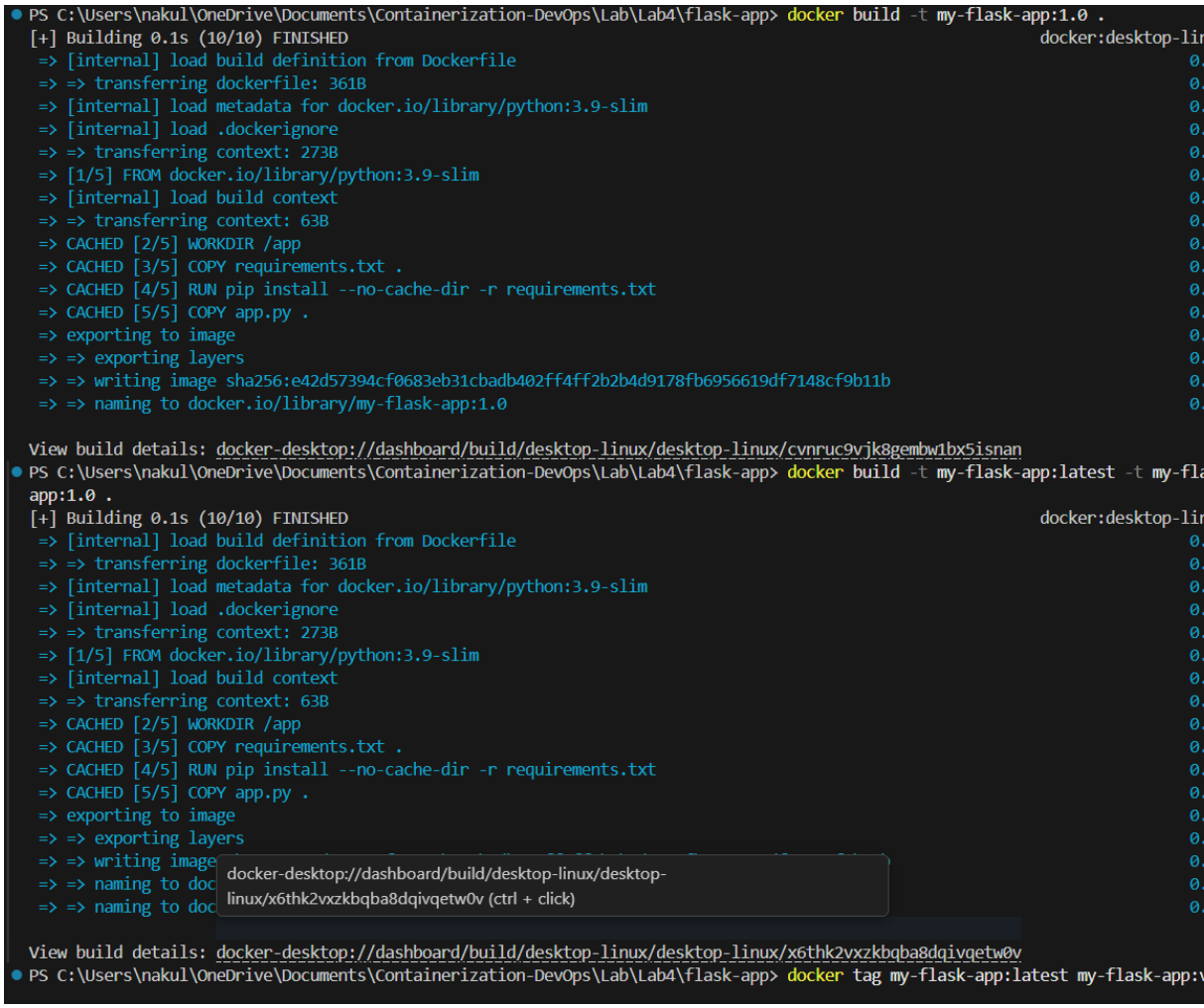
```
docker swarm init
```

Output:

```
Swarm initialized: current node (st2rbv9ix2c82um3rvm7ajijo) is now a manager.

To add a worker to this swarm, run the following command:
    docker swarm join --token SWMTKN-1-2wlkqwjnrdeqrcngjja2k856oh6ov1nau9t5bpkxbecvsh2lze-9gxka
```

This machine became the **manager node** of a single-node Swarm cluster.

Screenshot:



25.5.3 Task 3: Deploy as a Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

Output:

```
Ignoring unsupported options: restart
```

```
Ignoring deprecated options:
container_name: Setting the container name is not supported.

Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
```

> **Note:** Swarm ignored `restart` and `container_name` from the Compose file — these are Compose-only options. Swarm manages restarts automatically through its own service policies.

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
assignment1-backend:latest     696e1abfeb2b       185MB             0B
assignment1-db:latest          7068f884d33f       276MB             0B
experiment4:1.0                0e08867af4d9       128MB             0B
experiment4:latest             4231c3a2811e       132MB             0B    U
experiment:1.0                 0e08867af4d9       128MB             0B
flask-multistage:latest        94a39ca50667       145MB             0B
flask-regular:latest           4231c3a2811e       132MB             0B    U
my-flask-app:1.0               e42d57394cf0       132MB             0B
my-flask-app:latest            e42d57394cf0       132MB             0B
my-flask-app:v1.0              e42d57394cf0       132MB             0B
mysql:8                        a0f6c7786c75       777MB             0B    U
mysql:8.0                      6f19538dd7d2       780MB             0B
nakul71/my-flask-app:1.0       0e08867af4d9       128MB             0B
nginx-alpine:latest            a33d5a30a4a0      10.4MB             0B
nginx-ubuntu:latest            5363ea9a780d       207MB             0B
nginx:latest                   5cdef4ac3335       161MB             0B
python:3.9-slim                085da638e1b8       122MB             0B
ubuntu:latest                  493218ed0f40      78.1MB             0B
wordpress:latest               6fe8e9457b10       752MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker history my-flask-app
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e42d57394cf0   11 minutes ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY app.py . # buildkit                        228B      buildkit.dockerfile.v0
<missing>      11 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   10.5MB    buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY requirements.txt . # buildkit              13B       buildkit.dockerfile.v0
<missing>      3 weeks ago      WORKDIR /app                                    0B        buildkit.dockerfile.v0
<missing>      4 months ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  for src in idle3 p…   36B       buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…   39.5MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  apt-get update;  a…   3.81MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   78.6MB    debuerreotype 0.16
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
>> docker inspect my-flask-app
[
    {
        "Id": "sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b",
        "RepoTags": [
            "my-flask-app:1.0",
            "my-flask-app:latest",
            "my-flask-app:v1.0"
        ],
```

25.5.4 Task 4: Scale the Application

Scaled WordPress from 1 to 3 replicas:

```
docker service scale wpstack_wordpress=3
```

Output:

```
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
```

All 3 replicas started and Swarm confirmed convergence.

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker run -d -p 5000:5000 --name flask-cont
r my-flask-app
>>
6f01073577084341c00e87d2de6ff354c17dc102264f25649799bc31c2a11c49
```

> **How 3 containers share port 8080:** Swarm's internal load balancer listens on port 8080 once and distributes traffic to all 3 containers — no port conflicts.

25.5.5 Task 5: Test Self-Healing

Step 1: Listed all WordPress containers:

```
docker ps | grep wordpress
```

Three containers were running: - `wpstack_wordpress.3` — ID `38b5c1avf7io` - `wpstack_wordpress.2` — ID `rhqxd1bd6q2p` - `wpstack_wordpress.1` — ID `1bd27bf53b24`

Step 2: Killed one container to simulate a crash:

```
docker kill 1bd27bf53b24
```

Step 3: Checked service status:

```
docker service ps wpstack_wordpress
```

Output showed: - `wpstack_wordpress.1` → **Starting** (new container being created) - Old `wpstack_wordpress.1` (mirk9...) → **Shutdown / Failed** `"task: non-zero exit (137)"` - `wpstack_wordpress.2` → **Running** (unaffected) - `wpstack_wordpress.3` → **Running** (unaffected)

Swarm automatically replaced the killed container. Total replicas remained = 3 with **no manual intervention**.

Screenshot:

```
inerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:21 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> curl http://localhost:5000/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y


StatusCode        : 200
StatusDescription : OK
Content           : OK
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 2
                    Content-Type: text/html; charset=utf-8
                    Date: Fri, 20 Mar 2026 04:32:31 GMT
                    Server: Werkzeug/3.1.6 Python/3.9.25

                    OK
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Fri, 20 Mar
                    2026 04:32:31 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 2



PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED         STATUS         PORTS                                       NAMES
6f0107357708   my-flask-app   "python app.py"   7 minutes ago   Up 7 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

25.5.6 Task 6: Remove the Stack

```
docker stack rm wpstack
```

Output:

```
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Both services and the overlay network were removed cleanly.

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t flask-regular .
 => [internal] load .dockerignore
 => => transferring context: 273B
 => [1/5] FROM docker.io/library/python:3.9-slim
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b
 => => naming to docker.io/library/flask-regular

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hcwinfxyz9znl4zzathqwj838
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -f Dockerfile.multistage -t flask-multistage .
>>
[+] Building 8.2s (13/13) FINISHED                                                                          docker:desktop-li
 => [internal] load build definition from Dockerfile.multistage
 => => transferring dockerfile: 486B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load .dockerignore
 => => transferring context: 273B
 => [builder 1/5] FROM docker.io/library/python:3.9-slim
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [builder 2/5] WORKDIR /app
 => CACHED [builder 3/5] COPY requirements.txt .
 => [builder 4/5] RUN python -m venv /opt/venv
 => [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt
 => [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv
 => [stage-1 4/5] COPY app.py .
 => [stage-1 5/5] RUN useradd -m -u 1000 appuser
 => exporting to image
 => => exporting layers
 => => writing image sha256:15e019e88dc649d3371adfd7203641fbae971a4cb776c52bb56e64a89bde403a
 => => naming to docker.io/library/flask-multistage

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/hm4w1vg4rgjqp4u2ou41tekcr
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images | findstr flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      15e019e88dc6        145MB             0B
flask-regular:latest         e42d57394cf0        132MB             0B
my-flask-app:1.0             e42d57394cf0        132MB             0B
my-flask-app:latest          e42d57394cf0        132MB             0B
my-flask-app:v1.0            e42d57394cf0        132MB             0B
nakul71/my-flask-app:1.0     0e08867af4d9        128MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
```

25.6 6. Key Observations

25.6.1 Observation 1: Compose File Reuse

The **same YAML file** works for both Compose and Swarm:

Command	Mode
`docker compose up -d`	Compose (single host, no orchestration)
`docker stack deploy`	Swarm (orchestration enabled)

Swarm silently ignores Compose-only options like `restart` and `container_name`.

25.6.2 Observation 2: Containers vs Services

Concept	Meaning
Container	A single running instance
Service	A definition of how to run containers (image, replicas, etc.)

In Swarm, you manage **services**, not individual containers.

25.6.3 Observation 3: The Port Mystery Solved

In plain Compose, scaling WordPress to 3 would **fail** (port 8080 conflict). In Swarm, the load balancer listens on port 8080 **once** and routes traffic internally — no conflicts.

25.7 7. Quick Reference

```
# Initialize Swarm
docker swarm init

# Deploy stack
docker stack deploy -c docker-compose.yml <stack-name>

# List services
docker service ls

# Scale service
docker service scale <stack_service>=<replicas>

# See service tasks
docker service ps <service-name>

# Remove stack
docker stack rm <stack-name>

# Leave Swarm
docker swarm leave --force
```

25.8 8. Result

- Successfully initialized Docker Swarm on a single-node cluster (node ID: `st2rbv9ix2c82um3rvm7ajijo`).
- Deployed the WordPress + MySQL stack using `docker stack deploy -c docker-compose.yml wpstack`.
- Scaled WordPress to 3 replicas — Swarm handled load balancing automatically with `verify: Service wpstack_wordpress converged`.

- Demonstrated self-healing: killed container `1bd27bf53b24`, Swarm recreated `wpstack_wordpress.1` automatically without any manual intervention.
- Cleaned up the stack with `docker stack rm wpstack`.

25.9 9. Navigation

- ← Lab 10 — SonarQube
- → Lab 12
- All Labs

26 Experiment 12: Container Orchestration using Kubernetes

26.1 1. Objective

Learn why Kubernetes is used, its basic concepts, and how to deploy, scale, and self-heal apps using Kubernetes commands.

26.2 2. Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run your app
Large ecosystem	Many tools and plugins available
Cloud-native support	Works on AWS, Google Cloud, Azure, etc.

26.3 3. Core Kubernetes Concepts

Docker Concept	Kubernetes Equivalent	What it means
Container	**Pod**	Smallest unit in K8s; wraps one or more containers
Compose service	**Deployment**	Describes how your app should run (replicas, image, etc.)
Load balancing	**Service**	Exposes your app to the outside world or other pods

Docker Concept	Kubernetes Equivalent	What it means
Scaling	**ReplicaSet**	Ensures a certain number of pod copies are always running

---

26.4 4. Prerequisites

- `kubectl` installed
- k3d cluster running (`k3d-mycluster-server-0` — used in this experiment)

26.5 5. Steps Performed

26.5.1 Task 1: Create and Apply the Deployment & Service

Created `wordpress-deployment.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

Created `wordpress-service.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
```

```yaml
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

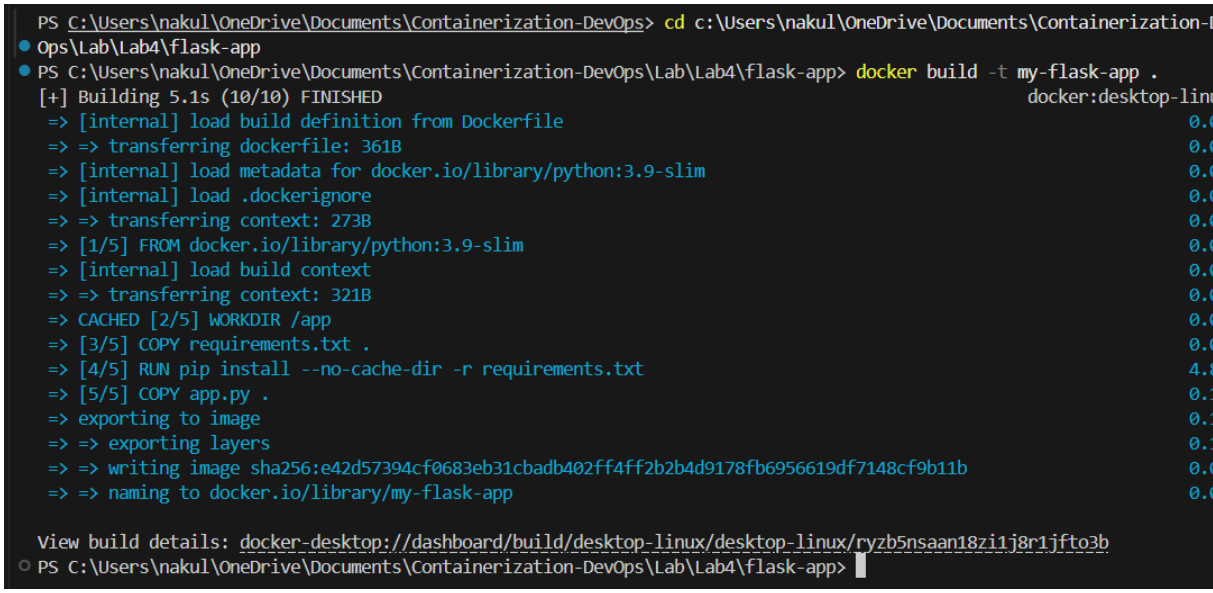
Applied both:

```
kubectl apply -f wordpress-deployment.yaml
kubectl apply -f wordpress-service.yaml
```

Output:

```
deployment.apps/wordpress created
service/wordpress-service created
```

Screenshot:



26.5.2 Task 2: Verify Pods and Service

```
kubectl get pods
kubectl get svc
```

Pod output:

```
NAME                          READY   STATUS              RESTARTS   AGE
wordpress-7d6f6db8d8-8xxwq    0/1     ContainerCreating   0          76s
wordpress-7d6f6db8d8-gtbsv    0/1     ImagePullBackOff    0          76s
```

Service output:

```
NAME                TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP   10.43.0.1      <none>        443/TCP        6m50s
wordpress-service   NodePort    10.43.15.190   <none>        80:30007/TCP   59s
```

Note: Pods showed `ContainerCreating` and `ImagePullBackOff` because the WordPress image was being pulled from Docker Hub in a limited network environment. The service was created and mapped correctly at `80:30007/TCP`.

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                    docker:desktop-lin
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 361B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load .dockerignore
 => => transferring context: 273B
 => [1/5] FROM docker.io/library/python:3.9-slim
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => writing image sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b
 => => naming to docker.io/library/my-flask-app:1.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cvnruc9vjk8gembw1bx5isnan
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker build -t my-flask-app:latest -t my-fla
app:1.0 .
[+] Building 0.1s (10/10) FINISHED                                                                    docker:desktop-lin
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 361B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load .dockerignore
 => => transferring context: 273B
 => [1/5] FROM docker.io/library/python:3.9-slim
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => writing image
 => => naming to doc   docker-desktop://dashboard/build/desktop-linux/desktop-
 => => naming to doc   linux/x6thk2vxzkbqba8dqivqetw0v (ctrl + click)

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/x6thk2vxzkbqba8dqivqetw0v
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker tag my-flask-app:latest my-flask-app:
```

26.5.3 Task 3: Scale the Deployment

Scaled WordPress from 2 to 4 replicas:

```
kubectl scale deployment wordpress --replicas=4
```

Output:

```
deployment.apps/wordpress scaled
```

Verified:

```
kubectl get pods
```

Output:

```
NAME                          READY   STATUS              RESTARTS   AGE
```

```
wordpress-7d6f6db8d8-8xxwq   0/1     ContainerCreating   0          2m21s
wordpress-7d6f6db8d8-gtbsv   0/1     ImagePullBackOff    0          2m21s
wordpress-7d6f6db8d8-jzt2j   0/1     ContainerCreating   0          5s
wordpress-7d6f6db8d8-wlqrv   0/1     ContainerCreating   0          5s
```

Kubernetes immediately created 2 additional pods (`jzt2j` and `wlqrv`) to reach the desired replica count of 4.

Screenshot:

```
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker images
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
assignment1-backend:latest     696e1abfeb2b        185MB             0B
assignment1-db:latest          7068f884d33f        276MB             0B
experiment4:1.0                0e08867af4d9        128MB             0B
experiment4:latest             4231c3a2811e        132MB             0B    U
experiment:1.0                 0e08867af4d9        128MB             0B
flask-multistage:latest        94a39ca50667        145MB             0B
flask-regular:latest           4231c3a2811e        132MB             0B    U
my-flask-app:1.0               e42d57394cf0        132MB             0B
my-flask-app:latest            e42d57394cf0        132MB             0B
my-flask-app:v1.0              e42d57394cf0        132MB             0B
mysql:8                        a0f6c7786c75        777MB             0B    U
mysql:8.0                      6f19538dd7d2        780MB             0B
nakul71/my-flask-app:1.0       0e08867af4d9        128MB             0B
nginx-alpine:latest            a33d5a30a4a0       10.4MB             0B
nginx-ubuntu:latest            5363ea9a780d        207MB             0B
nginx:latest                   5cdef4ac3335        161MB             0B
python:3.9-slim                085da638e1b8        122MB             0B
ubuntu:latest                  493218ed0f40       78.1MB             0B
wordpress:latest               6fe8e9457b10        752MB             0B
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app> docker history my-flask-app
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e42d57394cf0   11 minutes ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY app.py . # buildkit                        228B      buildkit.dockerfile.v0
<missing>      11 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   10.5MB    buildkit.dockerfile.v0
<missing>      11 minutes ago   COPY requirements.txt . # buildkit              13B       buildkit.dockerfile.v0
<missing>      3 weeks ago      WORKDIR /app                                    0B        buildkit.dockerfile.v0
<missing>      4 months ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  for src in idle3 p…   36B       buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   savedAptMark="$(a…   39.5MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3…   0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;  apt-get update;  a…   3.81MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   78.6MB    debuerreotype 0.16
PS C:\Users\nakul\OneDrive\Documents\Containerization-DevOps\Lab\Lab4\flask-app>
>> docker inspect my-flask-app
[
    {
        "Id": "sha256:e42d57394cf0683eb31cbadb402ff4ff2b2b4d9178fb6956619df7148cf9b11b",
        "RepoTags": [
            "my-flask-app:1.0",
            "my-flask-app:latest",
            "my-flask-app:v1.0"
        ],
```

26.5.4 Task 4: Self-Healing Demonstration

Step 1: Deleted one pod manually to simulate a crash:

`kubectl delete pod wordpress-7d6f6db8d8-8xxwq`

Output:

```
pod "wordpress-7d6f6db8d8-8xxwq" deleted from default namespace
```

Step 2: Inspected the deleted pod to observe Kubernetes self-healing:

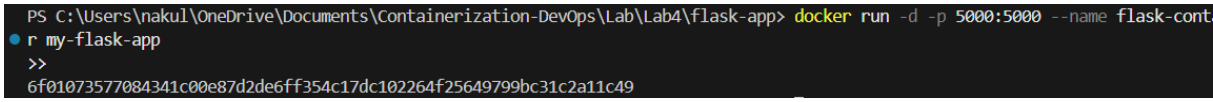
```
kubectl describe pod wordpress-7d6f6db8d8-8xxwq
```

Key details from the output:

```
Name:       wordpress-7d6f6db8d8-8xxwq
Namespace:  default
Node:       k3d-mycluster-server-0/172.24.0.3
Start Time: Fri, 01 May 2026 05:59:13 +0000
Labels:     app=wordpress
            pod-template-hash=7d6f6db8d8
Status:     Terminating (lasts 2m25s)
Controlled By: ReplicaSet/wordpress-7d6f6db8d8
State:      Waiting
  Reason:   ContainerCreating
```

> The pod was in `Terminating` state while Kubernetes simultaneously created a replacement — controlled by `ReplicaSet/wordpress-7d6f6db8d8`. Total replica count remained at 4 with no manual intervention.

Screenshot:



26.6 6. Swarm vs Kubernetes

Feature	Docker Swarm	Kubernetes
Setup	Very easy	More complex
Scaling	Basic	Advanced (auto-scaling)
Ecosystem	Small	Huge (monitoring, logging, etc.)
Industry use	Rare	Standard

Verdict: Kubernetes is the industry standard — used by AWS, GCP, Azure, and most companies.

26.7 7. Kubernetes Command Cheat Sheet

Goal	Command
Apply a YAML file	`kubectl apply -f file.yaml`
See all pods	`kubectl get pods`
See all services	`kubectl get svc`
Scale a deployment	`kubectl scale deployment <name> --replicas=N`

Goal	Command
Delete a pod	`kubectl delete pod <pod-name>`
Describe a resource	`kubectl describe pod <pod-name>`
See all nodes	`kubectl get nodes`

26.8 8. Result

- Successfully created a Kubernetes Deployment (`wordpress`) and exposed it via a NodePort Service on port `30007`.
- Scaled the deployment from 2 to 4 replicas — Kubernetes immediately created 2 additional pods.
- Demonstrated self-healing: deleted pod `wordpress-7d6f6db8d8-8xxwq`; Kubernetes automatically scheduled a replacement, maintaining the desired replica count of 4.
- Confirmed the cluster was running on k3d (`k3d-mycluster-server-0`), controlled by `ReplicaSet/wordpress-7d6f6db8d8`.

26.9 9. Navigation

- ← Lab 11 — Docker Swarm
- All Labs